



Technical University Munich  
School of Engineering and Design

# Report for Deep Learning for PDEs in Engineering Physics

Manuel Bryan Arsenio  
03789883

*Supervisors:* Dr. Yaohua Zang & Vincent Scholz

July 15, 2026

# Contents

|          |                                 |           |
|----------|---------------------------------|-----------|
| <b>1</b> | <b>Problem A</b>                | <b>1</b>  |
| 1.1      | Problem Description . . . . .   | 1         |
| 1.2      | Methodology . . . . .           | 1         |
| 1.2.1    | The method selection . . . . .  | 1         |
| 1.2.2    | The loss function . . . . .     | 2         |
| 1.2.3    | The network structure . . . . . | 3         |
| 1.2.4    | The code link . . . . .         | 3         |
| 1.3      | Numerical Experiments . . . . . | 4         |
| 1.3.1    | Experiment Setups . . . . .     | 4         |
| 1.3.2    | Numerical Results . . . . .     | 4         |
| <b>2</b> | <b>Problem B</b>                | <b>8</b>  |
| 2.1      | Problem Description . . . . .   | 8         |
| 2.2      | Methodology . . . . .           | 9         |
| 2.2.1    | The method selection . . . . .  | 9         |
| 2.2.2    | The test function . . . . .     | 9         |
| 2.2.3    | The loss function . . . . .     | 9         |
| 2.2.4    | The network structure . . . . . | 10        |
| 2.2.5    | The code link . . . . .         | 11        |
| 2.3      | Numerical Experiments . . . . . | 11        |
| 2.3.1    | Experiment Setups . . . . .     | 11        |
| 2.3.2    | Numerical Results . . . . .     | 12        |
| <b>3</b> | <b>Problem C</b>                | <b>14</b> |
| 3.1      | Problem Description . . . . .   | 14        |
| 3.2      | Methodology . . . . .           | 14        |
| 3.2.1    | The method selection . . . . .  | 14        |
| 3.2.2    | The loss function . . . . .     | 15        |
| 3.2.3    | The network structure . . . . . | 16        |
| 3.2.4    | The code link . . . . .         | 16        |
| 3.3      | Numerical Experiments . . . . . | 16        |
| 3.3.1    | Experiment Setups . . . . .     | 16        |
| 3.3.2    | Numerical Results . . . . .     | 17        |
| <b>4</b> | <b>Problem D</b>                | <b>20</b> |
| 4.1      | Problem Description . . . . .   | 20        |
| 4.2      | Methodology . . . . .           | 21        |
| 4.2.1    | Method selection . . . . .      | 21        |
| 4.2.2    | The loss function . . . . .     | 21        |

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| 4.2.3    | The network structure . . . . .                       | 22        |
| 4.2.4    | The code link . . . . .                               | 22        |
| 4.3      | Numerical Experiments . . . . .                       | 22        |
| 4.3.1    | Experiment Setups . . . . .                           | 22        |
| 4.3.2    | Numerical Results . . . . .                           | 23        |
| <b>5</b> | <b>Conclusions</b>                                    | <b>26</b> |
|          | <b>References</b>                                     | <b>27</b> |
|          | <b>Appendices</b>                                     | <b>28</b> |
| <b>A</b> | <b>Additional Hyperparameter Trials for Problem A</b> | <b>28</b> |
| <b>B</b> | <b>Additional Hyperparameter Trials for Problem B</b> | <b>30</b> |
| <b>C</b> | <b>Additional Hyperparameter Trials for Problem C</b> | <b>32</b> |
| <b>D</b> | <b>Additional Hyperparameter trials for Problem D</b> | <b>36</b> |

# Chapter 1

## Problem A

### 1.1 Problem Description

Problem A considers a one-dimensional static elastostatic problem on the interval  $(0, L)$ , with  $L = 1$ . The displacement field  $u(x)$  satisfies the governing equation

$$-\frac{d}{dx} \left( k(x) \frac{du}{dx} \right) = f, \quad x \in (0, L), \quad (1.1)$$

where  $k(x)$  is the Young's modulus and  $f = 9.81$  is the applied body force per unit length. The rod is fixed at both ends, giving the boundary conditions

$$u(0) = u(L) = 0. \quad (1.2)$$

The objective is to recover the unknown spatially varying Young's modulus  $k(x) > 0$  from noisy displacement observations. The available measurements consist of  $N_{\text{obs}} = 500$  randomly placed sensors  $x_{\text{obs}}$ , where the observed displacement values  $u_{\text{obs}}$  are contaminated by approximately 5% noise. Therefore, this is an inverse problem: instead of solving for  $u(x)$  using a known  $k(x)$ , the coefficient  $k(x)$  must be identified from partial and noisy observations of the displacement field.

The dataset is provided in `Datasets/ProblemA_dataset.h5`. The observation data  $\{x_{\text{obs}}, u_{\text{obs}}\}$  are used for training, while the test data  $\{x_{\text{test}}, u_{\text{test}}, k_{\text{test}}\}$  are used only for evaluation. During training, the relative  $L^2$  errors of both  $u(x)$  and  $k(x)$  are computed on the test set, but the test data are not used in the optimization.

### 1.2 Methodology

#### 1.2.1 The method selection

Problem A is naturally an inverse problem where the unknown Young's modulus  $k(x)$  must be recovered from noisy displacement observations of displacement field  $u(x)$  together with the governing elastostatic equation. An Inverse Physics-Informed Neural Networks (Inverse PINN) is chosen for Problem A. The PINN framework is suitable because it is explicitly designed for both forward and inverse PDE problems [6].

Weak-form approaches such as Inverse Weak Adversarial Networks (Inverse WAN) are also relevant. Inverse WAN enforces the PDE through its weak formulation and after integration by parts, the residual involves lower-order derivatives than the corresponding strong-form residual, which can reduce the differentiation burden in the loss calculation [12]. In addition, weak-form

methods are attractive when the classical strong solution is not sufficiently well-behaved, since weak solutions may exist even when classical solutions do not [12].

However, for the present problem, a strong-form Inverse PINN is more appropriate, as the problem is one-dimensional, the domain is geometrically simple, and the required strong-form residual can be evaluated directly using automatic differentiation. Therefore, the additional complexity of the WAN weak formulation is not necessary for this task.

### 1.2.2 The loss function

A weighted combination of a data-matching loss and a PDE residual loss is used to train the Inverse PINN. Following the standard PINN formulation, the data loss enforces agreement with the observed displacement values, while the physics loss enforces the governing differential equation at collocation points [6]. No separate boundary loss term is used, as the boundary conditions are imposed as hard constraints through the ansatz for  $u_\theta(x)$ .

The displacement field is represented as

$$u_\theta(x) = \sin(\pi x) N_\theta(x), \quad (1.3)$$

where  $N_\theta(x)$  is the raw output of the displacement network. This automatically satisfies

$$u_\theta(0) = u_\theta(1) = 0. \quad (1.4)$$

The Young's modulus is written as

$$k_\phi(x) = \text{softplus}(M_\phi(x)) + \varepsilon, \quad (1.5)$$

where  $M_\phi(x)$  is the raw output of the coefficient network and  $\varepsilon = 10^{-6}$ . The Softplus transformation is used to enforce the physical constraint  $k_\phi(x) > 0$ . According to the PyTorch documentation, Softplus is a smooth approximation to ReLU and can be used to constrain a model output to always be positive [5].

The data loss is defined using the noisy displacement observations:

$$\mathcal{L}_{\text{data}} = \frac{1}{N_{\text{obs}}} \sum_{i=1}^{N_{\text{obs}}} |u_\theta(x_i^{\text{obs}}) - u_i^{\text{obs}}|^2.$$

The PDE residual is

$$r_{\theta,\phi}(x) = -\frac{d}{dx} \left( k_\phi(x) \frac{du_\theta}{dx}(x) \right) - f,$$

with  $f = 9.81$ . The corresponding physics loss is evaluated at interior collocation points  $x_i^c$ :

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_c} \sum_{i=1}^{N_c} |r_{\theta,\phi}(x_i^c)|^2.$$

Therefore the total training objective is

$$\mathcal{L} = w_{\text{PDE}} \mathcal{L}_{\text{PDE}} + w_{\text{data}} \mathcal{L}_{\text{data}}.$$

The numerical values of the loss weights are given in Section 1.3.1.

### 1.2.3 The network structure

Two separate neural networks are used. The first network,  $N_\theta(x)$ , represents the displacement field before applying the hard boundary constraint. The second network,  $M_\phi(x)$ , represents the Young's modulus before applying the Softplus function. Both networks are fully connected multilayer perceptrons with tanh activation functions in the hidden layers.

The tanh activation was chosen because the PDE residual requires differentiating the neural network output with respect to the spatial coordinate. Smooth activation functions are therefore preferable in PINN models, whereas non-smooth activations such as ReLU can be problematic when higher-order or derivative-based residuals are used [6, 7]. The final architecture was selected after preliminary experiments with different coefficient-network sizes, as summarized in Appendix A. The displacement network was kept relatively small and stable, while the coefficient network was made slightly deeper and wider because  $k(x)$  contains stronger spatial variation than  $u(x)$ .

Table 1.1: Architecture of the displacement network  $N_\theta(x)$

| Layer          | Size / Neurons | Activation Function | Notes                    |
|----------------|----------------|---------------------|--------------------------|
| Input Layer    | 1              | –                   | Spatial coordinate $x$   |
| Hidden Layer 1 | 40             | tanh                | Fully connected          |
| Hidden Layer 2 | 40             | tanh                | Fully connected          |
| Hidden Layer 3 | 40             | tanh                | Fully connected          |
| Hidden Layer 4 | 40             | tanh                | Fully connected          |
| Output Layer   | 1              | –                   | Raw output $N_\theta(x)$ |

The physical displacement prediction, shown in Eq. (1.3), is not the raw network output itself due to the hard constraint boundary condition, as described in Section 1.2.2. The physical Young's modulus prediction, shown in Eq. (1.5), is obtained by applying a Softplus transformation to the raw output, as described in 1.2.2. This enforces the physical requirement  $k_\phi(x) > 0$ .

Table 1.2: Architecture of the Young's modulus network  $M_\phi(x)$

| Layer          | Size / Neurons | Activation Function | Notes                  |
|----------------|----------------|---------------------|------------------------|
| Input Layer    | 1              | –                   | Spatial coordinate $x$ |
| Hidden Layer 1 | 64             | tanh                | Fully connected        |
| Hidden Layer 2 | 64             | tanh                | Fully connected        |
| Hidden Layer 3 | 64             | tanh                | Fully connected        |
| Hidden Layer 4 | 64             | tanh                | Fully connected        |
| Hidden Layer 5 | 64             | tanh                | Fully connected        |
| Output Layer   | 1              | –                   | Raw output $M_\phi(x)$ |

### 1.2.4 The code link

[https://github.com/Manuel-Bryan-Arsenio/ED140023-Deep-Learning-for-PDEs-Project/blob/main/Problem\\_A\\_Results/Problem\\_A\\_Final.ipynb](https://github.com/Manuel-Bryan-Arsenio/ED140023-Deep-Learning-for-PDEs-Project/blob/main/Problem_A_Results/Problem_A_Final.ipynb)

## 1.3 Numerical Experiments

### 1.3.1 Experiment Setups

The final inverse PINN was implemented in PyTorch and trained using the Adam optimizer. Separate learning rates were assigned to the displacement network  $u_\theta$  and the coefficient network  $k_\phi$ , although both were set to the same initial value in the final experiment. The numerical setup is summarized in Table 1.3.

Table 1.3: Numerical setup for training the inverse PINN in PyTorch

| Parameter                            | Value                            |
|--------------------------------------|----------------------------------|
| Optimizer                            | Adam                             |
| Initial learning rate for $u_\theta$ | $1 \times 10^{-3}$               |
| Initial learning rate for $k_\phi$   | $1 \times 10^{-3}$               |
| Number of collocation points         | 10000                            |
| Batch Size                           | 1000                             |
| Number of Epochs                     | 25000                            |
| Learning Rate Scheduler              | StepLR                           |
| Weight decay                         | 0                                |
| Hardware                             | NVIDIA RTX 4050 Laptop GPU (6GB) |
| Precision                            | Float32                          |
| Early Stopping                       | No                               |

In the final experiment,  $N_{\text{obs}} = 500$  noisy observation points were used for the data loss, and  $N_c = 10000$  collocation points were sampled uniformly in the interval  $(0, 1)$  for the PDE loss. Mini-batch training was used for the collocation points with a batch size of 1000, while the observation data were evaluated in full batch. The final loss weights were chosen as

$$w_{\text{PDE}} = 2, \quad w_{\text{data}} = 200.$$

A larger weight was assigned to the data loss because the displacement observations provide the direct information for the inverse problem, while the PDE residual regularizes the recovered coefficient field through the governing equation.

Several implementation choices were used to improve the stability of the inverse recovery. First, the boundary conditions were imposed as hard constraints through the displacement ansatz in Eq. (1.3), so no additional boundary loss was required. Second, the Young's modulus was constrained to be positive using the Softplus transformation in Eq. (1.5). Third, weight decay was set to zero because additional parameter regularization was found to over-smooth the recovered coefficient field. Finally, the coefficient network was chosen to be larger than the displacement network, since  $k(x)$  contains stronger spatial variation than  $u(x)$ .

The final model was trained for 25000 epochs. This value was chosen based on preliminary training runs, summarized in Appendix A. In particular, the 20000-epoch run still showed a slowly decreasing coefficient error, while the 25000-epoch run gave a clearer plateau in the relative  $L^2$  error of  $k(x)$ . Since the main objective of the inverse problem is to recover  $k(x)$ , the final training duration was selected primarily based on the coefficient error rather than the displacement error.

### 1.3.2 Numerical Results

The final relative  $L^2$  errors on the test dataset were

$$e_u = 3.93 \times 10^{-3}, \quad e_k = 8.11 \times 10^{-2}.$$

Thus, the displacement field was reconstructed very accurately, while the Young's modulus was recovered with a larger but still reasonable error.

Figure 1.1 shows the recovered Young's modulus compared with the reference solution. The prediction captures the two high-stiffness regions and the boundary behavior of  $k(x)$ . However, the central low-stiffness region is slightly over-smoothed. This behavior is also visible in the pointwise absolute error shown in Fig. 1.2, where the largest error occurs near the center of the domain.

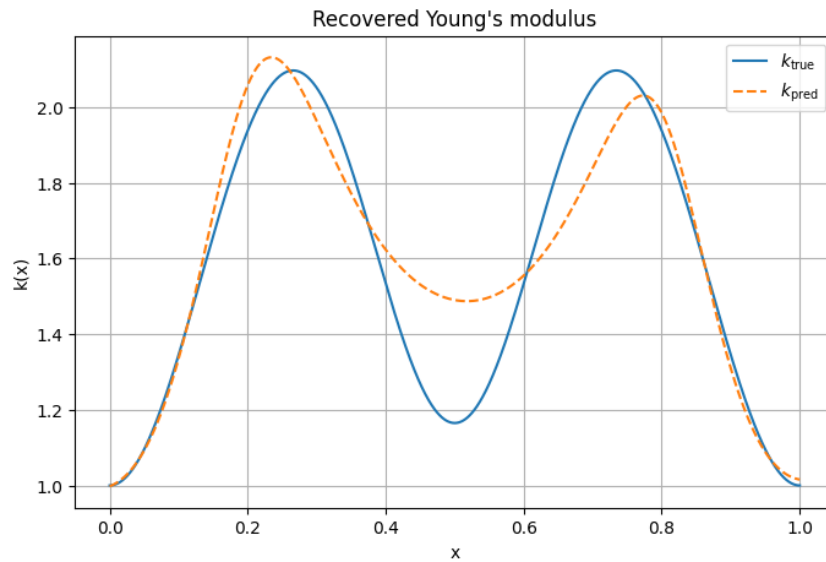


Figure 1.1: Recovered Young's modulus  $k_{\phi}(x)$  compared with the reference  $k_{\text{true}}(x)$ .

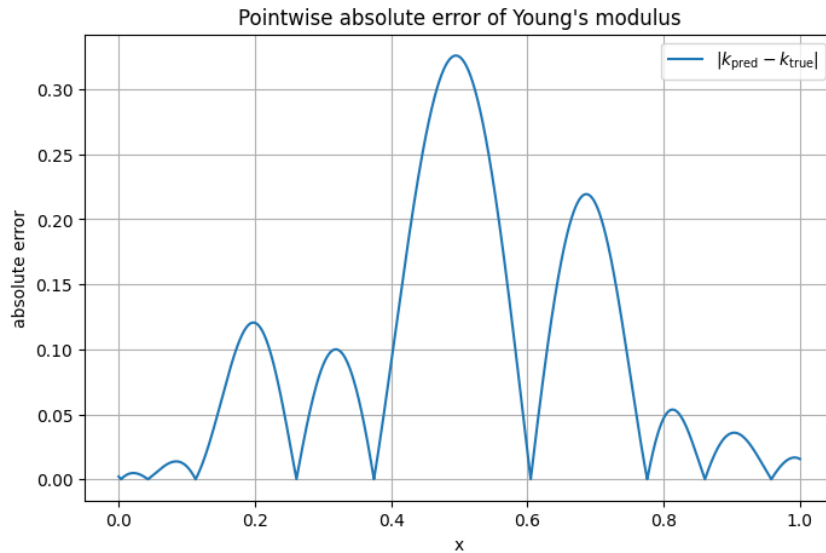


Figure 1.2: Pointwise absolute error of the recovered Young's modulus.

Figure 1.3 shows that the displacement prediction agrees closely with the reference displacement and follows the noisy observation data, consistent with the small relative  $L^2$  error.

Figure 1.5 shows that the  $L^2$  relative displacement error converges quickly, while the



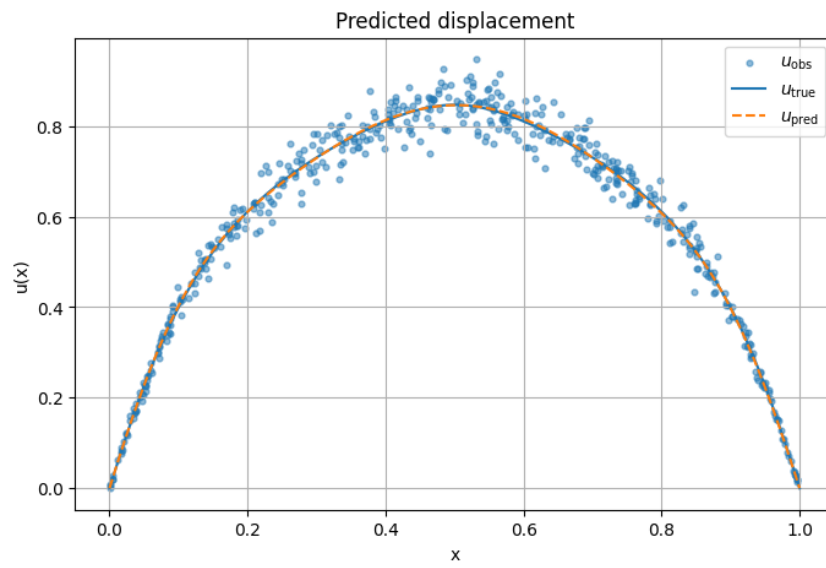


Figure 1.3: Predicted displacement  $u_{\theta}(x)$ , reference displacement  $u_{\text{true}}(x)$ , and noisy observations  $u_{\text{obs}}$ .

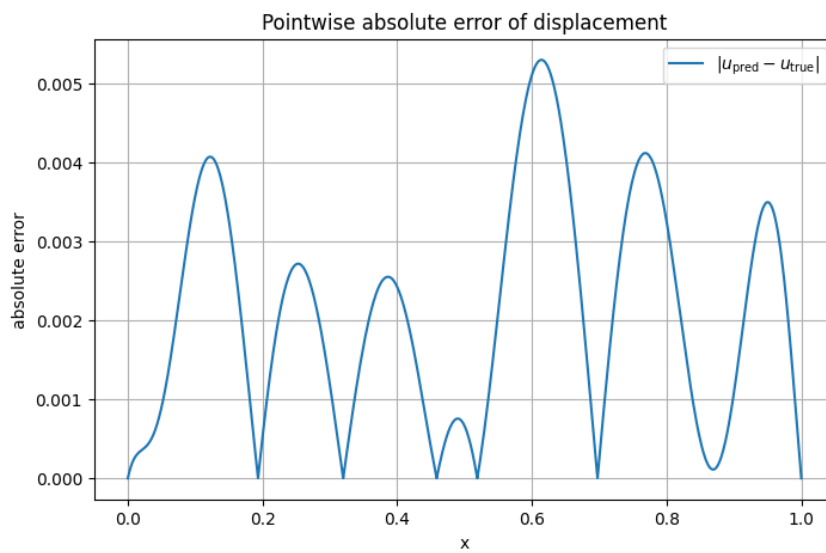


Figure 1.4: Pointwise absolute error of the displacement  $u_{\theta}(x)$ .

coefficient error decreases more slowly and stabilizes near the end of training. This confirms that recovering  $k(x)$  is the more difficult part of the inverse problem.

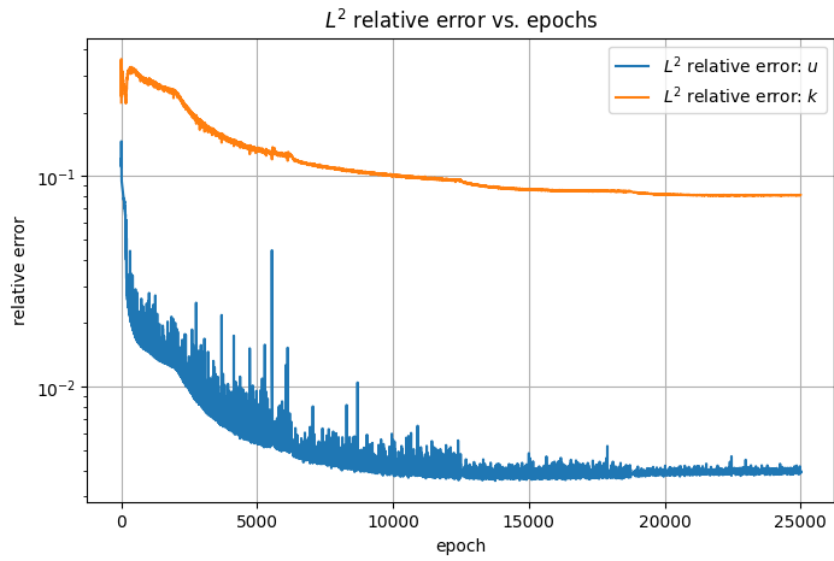


Figure 1.5: Relative  $L^2$  errors of the displacement  $u(x)$  and Young's modulus  $k(x)$  during training.

## Chapter 2

# Problem B

### 2.1 Problem Description

Problem B considers a square domain  $\Omega = [0, 1]^2$  representing a heterogeneous porous medium composed of two material phases with different permeabilities. The permeability field is given by

$$\mu(x_1, x_2) = \begin{cases} \mu_1 = 10, & (x_1, x_2) \in \Omega_1, \\ \mu_2 = 2, & (x_1, x_2) \in \Omega_2, \end{cases}$$

where  $\Omega_1$  denotes the high-permeability phase (e.g., sandstone) and  $\Omega_2$  denotes the low-permeability phase (e.g., mudstone). The exact geometric distribution of the two phases is provided in the dataset as a  $128 \times 128$  permeability matrix.

The pressure field  $u(x_1, x_2)$  satisfies the steady Darcy equation

$$-\nabla \cdot (\mu(x_1, x_2) \nabla u) = f, \quad (x_1, x_2) \in \Omega, \quad (2.1)$$

where  $f = 0$ , i.e., no internal source or sink. The boundary conditions prescribe a high pressure on the left boundary and a low pressure on the right boundary:

$$u(x_1, x_2) = g_D(x_1, x_2), \quad (2.2)$$

with

$$g_D(x_1, x_2) = \begin{cases} 1, & x_1 = 0, \\ 0, & x_1 = 1, \\ 1 - x_1, & x_2 = 0 \text{ or } x_2 = 1. \end{cases}$$

Thus, the imposed pressure difference drives flow from left to right through the heterogeneous medium.

The goal is to compute the pressure field  $u(x_1, x_2)$  with the given permeability field  $\mu(x_1, x_2)$ . This is a forward problem, as the coefficient field  $\mu(x_1, x_2)$  and the boundary conditions are known. The dataset `Datasets/ProblemB_dataset.h5` provides the permeability field `mu_field`, together with test points `x_test` and the reference pressure values `u_test`. The test data are used only for evaluating the relative  $L^2$  error and are not used during training.

## 2.2 Methodology

### 2.2.1 The method selection

Problem B is a forward problem, since the permeability field  $\mu(x_1, x_2)$ , the governing Darcy equation, and the boundary conditions are already known. The goal is to compute the pressure field  $u(x_1, x_2)$ . Since  $\mu(x_1, x_2)$  is piecewise constant and has discontinuities at the material interfaces, a weak-form method is more suitable than a strong-form PINN. A strong-form PINN enforces the PDE residual pointwise, while a weak-form method can reduce the regularity requirement and lower the derivative order by integration by parts [1].

For this reason, a Variational Physics-Informed Neural Network (VPINN) is selected. VPINN can be understood as a Petrov–Galerkin version of PINN, where the trial function is represented by a neural network and the test functions are chosen separately, for example as polynomial functions [1]. This fits the Darcy equation well as the weak residual uses  $\mu(x_1, x_2)$  and  $\nabla u_\theta(x_1, x_2)$ , but does not require taking derivatives of the discontinuous permeability field.

Deep Ritz is also a possible choice, since the Darcy equation can also be written as an energy minimization problem. However, VPINN is chosen because it enforces the PDE through weak residuals with test functions, which is closer to the variational formulation discussed in the VPINN method [1]. WAN and ParticleWNN are also relevant weak-form methods, but they are more complex for this problem. WAN requires adversarial minimax training [12], while ParticleWNN requires choices of particle centers, compact support radii, and local test functions [11]. Therefore, VPINN is chosen as a simpler weak-form method that is still suitable for the discontinuous permeability field.

### 2.2.2 The test function

The test functions are built from Legendre polynomials, as shown in Eq. (2.3). Legendre polynomials are orthogonal, which provides maximally independent information between different polynomial modes; has global support, allowing each function to probe the entire domain; and is analytically tractable through known recurrence and derivative relations. The test function is constructed such that it vanishes on  $\partial\Omega$  [1]. This eliminates the unknown boundary-flux term arising from integration by parts. Additionally, integration by parts reduces the differential order applied to the neural-network solution, such that the weak residual only requires its first derivatives, while the prescribed Dirichlet conditions are enforced separately.

Since Legendre polynomials are orthogonal on the reference interval  $[-1, 1]$ , the physical coordinates  $x_1, x_2 \in [0, 1]$  are mapped using  $\xi = 2x_1 - 1$  and  $\eta = 2x_2 - 1$ .

$$v_{mn}(x_1, x_2) = [P_{m+1}(\xi) - P_{m-1}(\xi)][P_{n+1}(\eta) - P_{n-1}(\eta)], \quad m, n = 1, \dots, M. \quad (2.3)$$

Here,  $P_k$  denotes the Legendre polynomial of degree  $k$ , defined recursively by

$$P_0(z) = 1, \quad P_1(z) = z, \quad P_{k+1}(z) = \frac{(2k+1)zP_k(z) - kP_{k-1}(z)}{k+1}, \quad k \geq 1. \quad (2.4)$$

### 2.2.3 The loss function

A weighted combination of a weak-form PDE residual loss and a boundary loss is used to train the VPINN. The governing equation is multiplied by test functions and enforced through the weak residual. This is suitable for this problem because the permeability field is discontinuous, so the weak formulation avoids taking derivatives of  $\mu(x_1, x_2)$ .

For each Legendre test function  $v_{mn}$ , the weak residual is written as

$$R_{mn}(u_\theta) = \int_{\Omega} \mu(x_1, x_2) \left( \frac{\partial u_\theta}{\partial x_1} \frac{\partial v_{mn}}{\partial x_1} + \frac{\partial u_\theta}{\partial x_2} \frac{\partial v_{mn}}{\partial x_2} \right) d\Omega.$$

Since the source term in Problem B is  $f = 0$ , there is no additional right-hand side term in the residual. The corresponding PDE loss is defined as the mean squared weak residual over all test functions:

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_v} \sum_{m,n} |R_{mn}(u_\theta)|^2,$$

where  $N_v$  is the total number of test functions.

In the implementation, the weak integral is evaluated using the grid points associated with the provided  $128 \times 128$  permeability field. Mini-batch training is used for the interior integral points. Therefore, for a mini-batch with  $N_b$  points, the residual is approximated as

$$R_{mn}(u_\theta) \approx \frac{1}{N_b} \sum_{i=1}^{N_b} \mu(x_{1,i}, x_{2,i}) (u_{\theta,x_1}(x_{1,i}, x_{2,i}) v_{mn,x_1}(x_{1,i}, x_{2,i}) + u_{\theta,x_2}(x_{1,i}, x_{2,i}) v_{mn,x_2}(x_{1,i}, x_{2,i})).$$

The permeability value  $\mu(x_{1,i}, x_{2,i})$  is taken directly from the given `mu_field`. Thus, the permeability field is used during training as the known coefficient in the PDE.

The boundary condition is imposed using a separate soft boundary loss. The prescribed Dirichlet boundary condition is

$$g_D(x_1, x_2) = 1 - x_1.$$

Therefore, the boundary loss is

$$\mathcal{L}_{\text{BD}} = \frac{1}{N_{\text{BD}}} \sum_{i=1}^{N_{\text{BD}}} |u_\theta(x_{1,i}^{\text{BD}}, x_{2,i}^{\text{BD}}) - g_D(x_{1,i}^{\text{BD}}, x_{2,i}^{\text{BD}})|^2.$$

The total loss function is then

$$\mathcal{L} = w_{\text{PDE}} \mathcal{L}_{\text{PDE}} + w_{\text{BD}} \mathcal{L}_{\text{BD}}.$$

The numerical values of  $N_{\text{test}}$ , the mini-batch size, and the loss weights are given in Section 2.3.1. The preliminary runs used to select these values are summarized in Appendix B.

## 2.2.4 The network structure

One neural network is used  $u_\theta(x_1, x_2)$ . The network takes the two spatial coordinates  $(x_1, x_2)$  as input and outputs the scalar pressure value. Since Problem B is a forward problem, the permeability field  $\mu(x_1, x_2)$  is not represented by a neural network. Instead, it is directly taken from the given `mu_field` data and used as a known coefficient in the PDE residual.

The pressure network is chosen as a fully connected multilayer perceptron with tanh activation functions in the hidden layers. As VPINN loss requires differentiating the neural network output with respect to the spatial coordinates, a smooth activation functions, such as tanh, are therefore preferable for derivative-based physics-informed losses, while non-smooth activations such as ReLU can be problematic when spatial derivatives are used [6, 7]. This is also consistent with the common choice in PINN and VPINN implementations [6, 1].

The final architecture uses four hidden layers with 64 neurons in each layer. This architecture was selected based on the preliminary experiments summarized in Appendix B. The final pressure network architecture is summarized in Table 2.1.

Table 2.1: Architecture of the pressure neural network for Problem B

| Layer          | Size / Neurons | Activation Function | Notes                                    |
|----------------|----------------|---------------------|------------------------------------------|
| Input Layer    | 2              | –                   | Spatial coordinates $(x_1, x_2)$         |
| Hidden Layer 1 | 64             | tanh                | Fully connected                          |
| Hidden Layer 2 | 64             | tanh                | Fully connected                          |
| Hidden Layer 3 | 64             | tanh                | Fully connected                          |
| Hidden Layer 4 | 64             | tanh                | Fully connected                          |
| Output Layer   | 1              | –                   | Pressure prediction $u_\theta(x_1, x_2)$ |

### 2.2.5 The code link

[https://github.com/Manuel-Bryan-Arsenio/ED140023-Deep-Learning-for-PDEs-Project/blob/main/Problem\\_B\\_Results/Problem\\_B\\_Result\\_5\\_final.ipynb](https://github.com/Manuel-Bryan-Arsenio/ED140023-Deep-Learning-for-PDEs-Project/blob/main/Problem_B_Results/Problem_B_Result_5_final.ipynb)

## 2.3 Numerical Experiments

### 2.3.1 Experiment Setups

The VPINN model was implemented in PyTorch and trained using the Adam optimizer. The pressure field  $u_\theta$  is represented by one fully connected neural network, while the permeability field  $\mu(x_1, x_2)$  is taken directly from the given dataset and used as a known coefficient. The numerical setup is summarized in Table 2.2.

Table 2.2: Numerical Setup for Training the VPINN in PyTorch

| Parameter                            | Value                            |
|--------------------------------------|----------------------------------|
| Optimizer                            | Adam                             |
| Initial learning rate for $u_\theta$ | $1 \times 10^{-3}$               |
| Number of integral points            | $128^2 = 16384$                  |
| Batch Size                           | 1000                             |
| Number of Epochs                     | 1500                             |
| Learning Rate Scheduler              | StepLR                           |
| Weight decay                         | 0                                |
| Hardware                             | NVIDIA RTX 4050 Laptop GPU (6GB) |
| Precision                            | Float32                          |
| Early Stopping                       | No                               |

In the final experiment,  $N_{\text{test}} = 6$  Legendre modes were used in each coordinate direction, hence,  $N_v = 36$  test functions in total. The weak-form PDE loss was evaluated using mini-batches of 1000 interior integral points, while the boundary loss was evaluated using all boundary points in each training iteration. The full set of interior points contains  $128^2 = 16384$  points, corresponding to the resolution of the provided permeability field. The loss weights are chosen as

$$w_{\text{PDE}} = 1, \quad w_{\text{BD}} = 50.$$

A soft boundary constraint is chosen. The target boundary value was computed from the prescribed Dirichlet condition  $g_D(x_1, x_2) = 1 - x_1$ . A hard-boundary ansatz was also tested, but it led to a larger final relative  $L^2$  error in the numerical experiments. Therefore, the soft-boundary formulation was retained for the final VPINN setup. The detailed preliminary runs, including the hard-boundary tests, are summarized in Appendix B.

The final model was trained for 1500 epochs. This value was chosen based on the preliminary training runs in Appendix B. In particular, longer training did not consistently reduce the final relative  $L^2$  error, while the selected setup reached a stable error plateau within 1500 epochs, as shown in Fig. 2.3 figure here]. Therefore, 1500 epochs were used for the final result.

### 2.3.2 Numerical Results

The final relative  $L^2$  error on the test dataset is

$$e_u = 8.61 \times 10^{-2}.$$

Thus, the pressure field was reconstructed accurately with a reasonable error. The predicted pressure field is shown in Fig. 2.1b. As seen in the figure, the main pressure decrease from the high-pressure left boundary to the low-pressure right boundary is captured well. The predicted pressure field has the correct global behavior, with pressure values close to one near the left boundary and close to zero near the right boundary.

When compared with the reference solution shown in Fig. 2.1a, the predicted pressure field is smoother. These local features are only partially captured by the VPINN prediction. This is also visible in the pointwise absolute error in Fig. 2.2, where most of the higher pointwise errors lies in in region affected by the heterogenous material distribution.

The relative  $L^2$  error history is shown in Fig. 2.3. The error decreases quickly during the first few hundred epochs and then stabilizes around  $8.6 \times 10^{-2}$ , suggesting that the remaining error is not caused by insufficient training time.

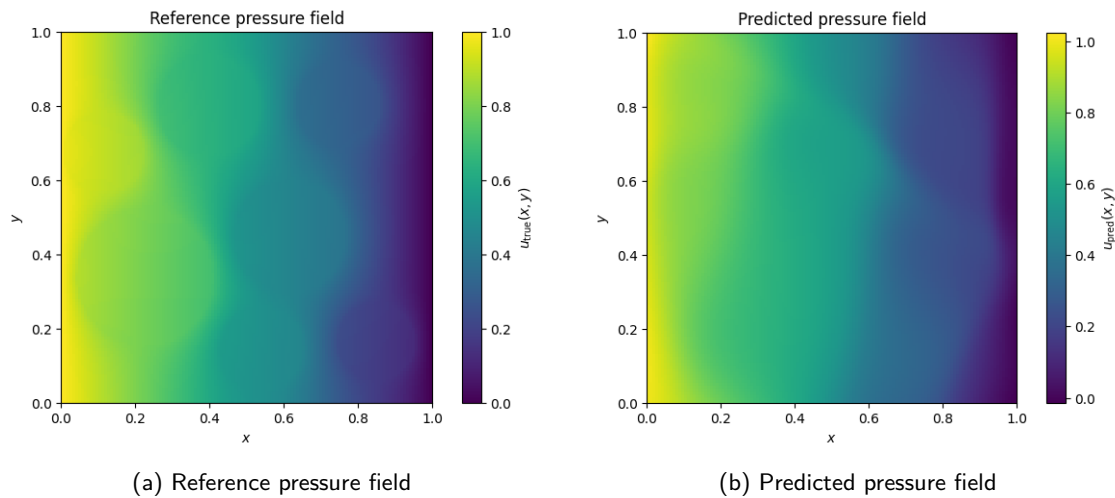


Figure 2.1: Reference and predicted pressure fields for Problem B. The VPINN captures the main left-to-right pressure decrease, but the predicted field is smoother than the reference solution.

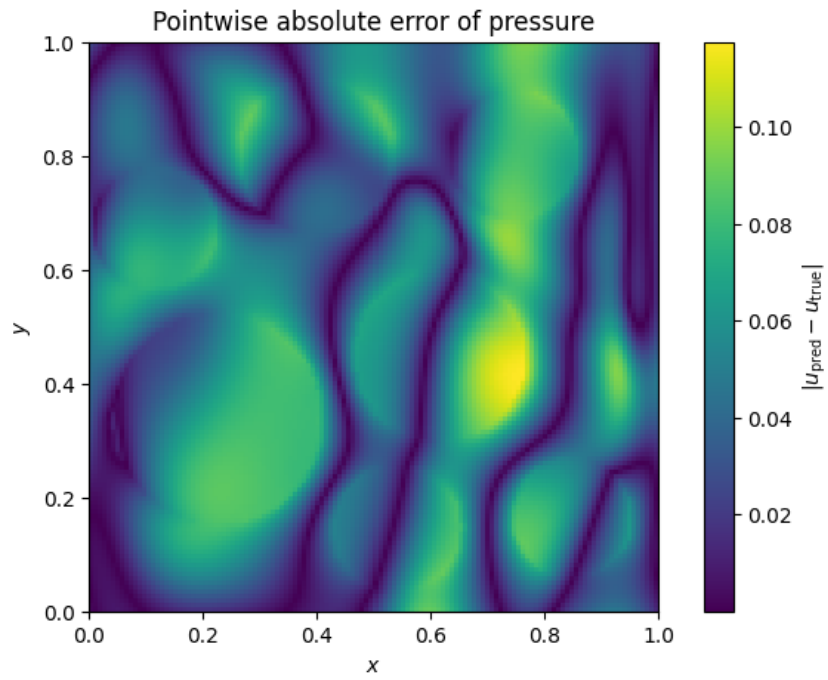


Figure 2.2: Pointwise absolute error  $|u_{\text{pred}} - u_{\text{true}}|$  for the final VPINN prediction.

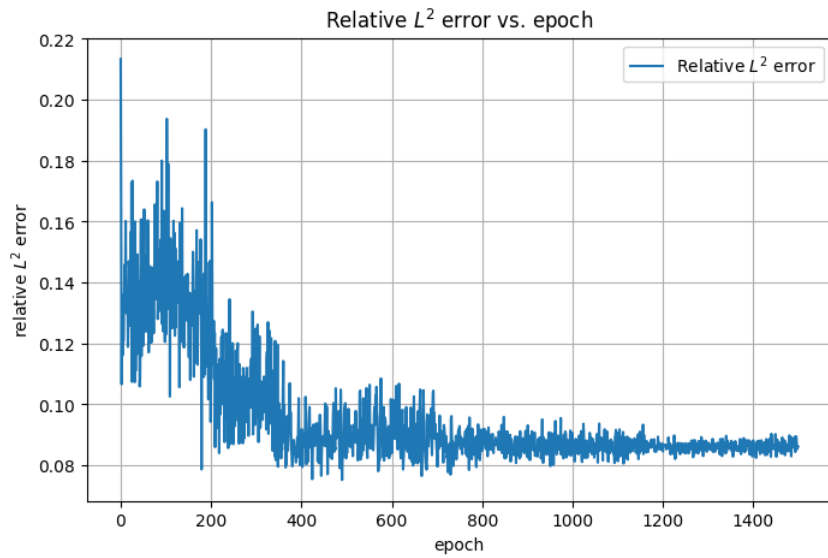


Figure 2.3: Relative  $L^2$  error of the pressure field during training.



## Chapter 3

# Problem C

### 3.1 Problem Description

Problem C considers a steady-state heat conduction problem on the square domain  $\Omega = [0, 1]^2$ . The material is heterogeneous, meaning that thermal conductivity is spatially varying. For each conductivity field  $a(x_1, x_2) > 0$ , the temperature field  $u(x_1, x_2)$  satisfies the governing equation

$$-\nabla \cdot (a(x_1, x_2) \nabla u) = f, \quad (x_1, x_2) \in \Omega, \quad (3.1)$$

with homogeneous Dirichlet boundary conditions

$$u(x_1, x_2) = 0, \quad (x_1, x_2) \in \partial\Omega. \quad (3.2)$$

The uniform heat source is given by  $f = 10$ . The zero Dirichlet boundary condition means that the boundary of the domain is kept at a fixed reference temperature.

The objective of this problem is to learn the solution operator that maps a conductivity field to the corresponding temperature field,

$$a(x_1, x_2) \mapsto u(x_1, x_2).$$

Therefore, the model has to learn from many input-output pairs and predict the temperature field for new, unseen conductivity fields. This makes Problem C an operator-learning problem.

The dataset `Datasets/ProblemC_dataset.h5` contains 1000 training samples and 200 test samples. The training data are given by the conductivity fields `a_train` and the corresponding FEM-computed temperature fields `u_train`. The test data are given by `a_test` and `u_test`, which are not used in the training process, only used to evaluate the prediction accuracy on unseen conductivity samples, i.e., compute the relative  $L^2$  error. The coordinate grids are provided as `X` and `Y`, both with a resolution of  $128 \times 128$ . During training, the model learns the mapping from the conductivity field to the temperature field.

### 3.2 Methodology

#### 3.2.1 The method selection

Problem C is an operator-learning problem, since the goal is to learn the mapping from the conductivity field to the corresponding temperature field,

$$G : a(x_1, x_2) \mapsto u(x_1, x_2).$$

As many input-output pairs are available, and the trained model should predict the solution for new unseen conductivity fields, a neural operator method is the natural choice. This follows the neural-operator setting discussed in the lecture, where the goal is to learn a function-to-function map for parametric PDEs [10, 8].

A Fourier Neural Operator (FNO) is selected for this problem. DeepONet is also a strong candidate for operator learning, but in this problem the input is the full conductivity field and the output is the full temperature field. Therefore, the problem is a full-field to full-field mapping, instead of point-to-point query, therefore, FNO is the more suitable method [8, 2]. In addition, the conductivity and temperature fields are both given on the same structured grid. FNO is designed for supervised operator learning on grid-based PDE data and uses Fourier layers to capture global spatial information [8]. This fits Problem C because both the conductivity and temperature fields are given on the same structured grid.

Physics-informed operator methods, such as PI-DeepONet or PINO, are also possible alternatives. These methods add the governing PDE into the training loss, which can be useful when labelled solution data are scarce or when one wants to enforce physics more directly [9, 3]. However, in this problem 1000 FEM-computed training pairs are given. Therefore, a supervised data-driven FNO is simpler and sufficient. Adding a physics-informed loss would increase the implementation complexity and training cost, while the available labelled data already provide direct supervision for the operator  $a \mapsto u$ .

### 3.2.2 The loss function

The loss function is based only on the mismatch between the predicted temperature field and the FEM-computed reference temperature field. However, no PDE residual loss, boundary loss, collocation points, or integral points are used. Hence, the physics is enforced only from the FEM-computed reference temperature field. Additionally, all reference FEM solutions satisfy the homogeneous Dirichlet boundary condition.

For each training sample, the FNO takes the conductivity field and coordinate grid as input,

$$[a(x_1, x_2), x_1, x_2],$$

and predicts the corresponding temperature field  $u_\theta(x_1, x_2)$ . The supervised data loss is computed between the predicted field and the FEM reference solution  $u_{\text{FEM}}(x_1, x_2)$ . In the implementation, both the input conductivity field and the output temperature field are normalized before training. Therefore, the loss is evaluated in normalized space:

$$\mathcal{L}_{\text{data}} = \frac{1}{N_b} \sum_{i=1}^{N_b} \left\| \tilde{u}_\theta^{(i)} - \tilde{u}_{\text{FEM}}^{(i)} \right\|_2,$$

where  $N_b$  is the batch size, and the tilde denotes normalized quantities. Each sample  $i$  corresponds to one full  $128 \times 128$  field, not to a single spatial point.

The relative  $L^2$  error is computed on the test set after each epoch. For one test sample, it is defined as

$$e_u^{(i)} = \frac{\left\| u_\theta^{(i)} - u_{\text{FEM}}^{(i)} \right\|_2}{\left\| u_{\text{FEM}}^{(i)} \right\|_2}.$$

The reported test error is the mean relative  $L^2$  error over all 200 test samples:

$$e_u = \frac{1}{200} \sum_{i=1}^{200} e_u^{(i)}.$$

The test error is computed after converting the prediction back to the original physical scale. Since the model only uses one supervised data loss, no loss weights are needed. The main training choices, such as the batch size, number of Fourier modes, hidden width, learning rate, and number of epochs, are discussed further in Section 3.3.1.

### 3.2.3 The network structure

The network used for Problem C is a two-dimensional Fourier Neural Operator. Each input sample is a full  $128 \times 128$  field with three channels,

$$[a(x_1, x_2), x_1, x_2],$$

where  $a(x_1, x_2)$  is the conductivity field and  $x_1, x_2$  are the coordinate grids. The output is one channel, corresponding to the predicted temperature field  $u_\theta(x_1, x_2)$ .

As shown in Table 3.1, the model first maps the input from 3 channels to a hidden width of 32. It then applies four Fourier blocks, where each block combines a spectral convolution with a pointwise  $1 \times 1$  convolution, followed by a GELU activation. Finally, the hidden representation is projected back to one output channel. In the final selected model, 20 Fourier modes are kept in both spatial directions. The Fourier modes and width are chosen iteratively summarized in Appendix C and discussed further in Section 3.3.1.

Table 3.1: Architecture of the FNO model for Problem C

| Part               | Size / Channels           | Activation | Notes                           |
|--------------------|---------------------------|------------|---------------------------------|
| Input field        | $128 \times 128 \times 3$ | –          | $[a, x_1, x_2]$                 |
| Lifting layer      | $3 \rightarrow 32$        | –          | Pointwise fully connected layer |
| Fourier blocks     | $32 \rightarrow 32$       | GELU       | 4 blocks, $m_1 = m_2 = 20$      |
| Projection layer 1 | $32 \rightarrow 128$      | GELU       | Pointwise fully connected layer |
| Projection layer 2 | $128 \rightarrow 1$       | –          | Predicted temperature field     |
| Output field       | $128 \times 128 \times 1$ | –          | $u_\theta(x_1, x_2)$            |

### 3.2.4 The code link

[https://github.com/Manuel-Bryan-Arsenio/ED140023-Deep-Learning-for-PDEs-Project/blob/main/Problem\\_C\\_Results/Problem\\_C\\_Result\\_4\\_final.ipynb](https://github.com/Manuel-Bryan-Arsenio/ED140023-Deep-Learning-for-PDEs-Project/blob/main/Problem_C_Results/Problem_C_Result_4_final.ipynb)

## 3.3 Numerical Experiments

### 3.3.1 Experiment Setups

The final FNO model was trained using the 1000 training samples from the dataset. The 200 test samples were not used during training and were only used to compute the relative  $L^2$  error after each epoch. Each sample corresponds to one full  $128 \times 128$  conductivity field and its corresponding FEM temperature field.

Before training, the conductivity input and temperature output were normalized using the mean and standard deviation of the training data. The coordinate channels  $x_1$  and  $x_2$  were not normalized, since they already lie in the interval  $[0, 1]$ . The model was trained using the Adam optimizer with a learning rate of  $10^{-3}$  and weight decay  $10^{-4}$ . A StepLR scheduler was used to reduce the learning rate during training. The batch size was 10, which means that each training batch contains 10 full field samples.

The final selected setup uses 20 Fourier modes in both spatial directions and a hidden width of 32. This setup was chosen based on the additional hyperparameter runs shown in Appendix C, where increasing the number of Fourier modes was more effective than increasing the hidden width.

Table 3.2: Numerical Setup for Training the FNO in PyTorch

| Parameter               | Value                             |
|-------------------------|-----------------------------------|
| Optimizer               | Adam                              |
| Initial learning rate   | $1 \times 10^{-3}$                |
| Batch size              | 10                                |
| Number of epochs        | 500                               |
| Training samples        | 1000                              |
| Test samples            | 200                               |
| Grid resolution         | $128^2 = 16384$ points per sample |
| Fourier modes           | $m_1 = m_2 = 20$                  |
| Hidden width            | 32                                |
| Learning rate scheduler | StepLR                            |
| Weight decay            | $1 \times 10^{-4}$                |
| Hardware                | NVIDIA RTX 4050 Laptop GPU (6GB)  |
| Precision               | Float32                           |
| Early stopping          | No                                |

### 3.3.2 Numerical Results

The total training time was 1 h 37 min 38 s on an NVIDIA RTX 4050 Laptop GPU, giving final mean relative  $L^2$  error over the 200 test samples was

$$e_u = 1.855718 \times 10^{-3}.$$

This is the best result among the tested configurations. As shown in Appendix C, increasing the number of Fourier modes improved the accuracy more efficiently than increasing the hidden width.

Figure 3.2b shows the prediction result for the first test sample. The predicted temperature field closely matches the FEM reference solution. The largest pointwise errors are mainly located in regions where the conductivity field has stronger spatial oscillations. This is expected, since these regions contain higher-frequency information and are therefore harder to approximate. Nevertheless, the maximum pointwise error remains small, and the overall relative  $L^2$  error is below  $2 \times 10^{-3}$ .

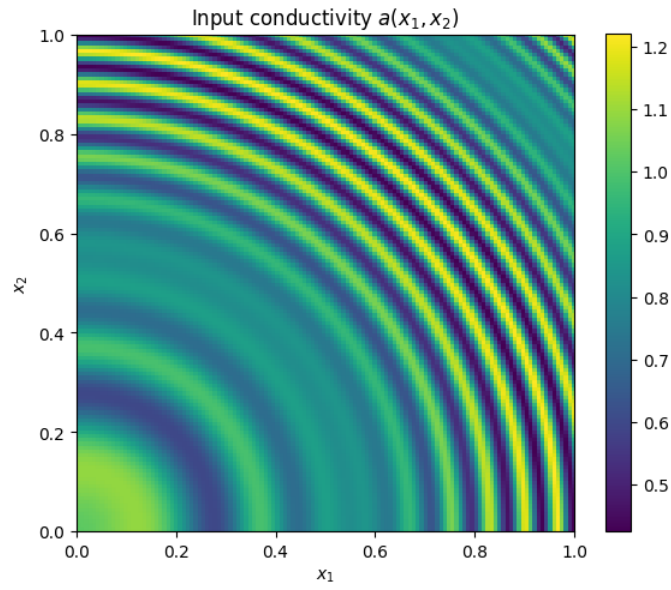


Figure 3.1: Input conductivity field  $a(x_1, x_2)$  for test sample 1 in Problem C.

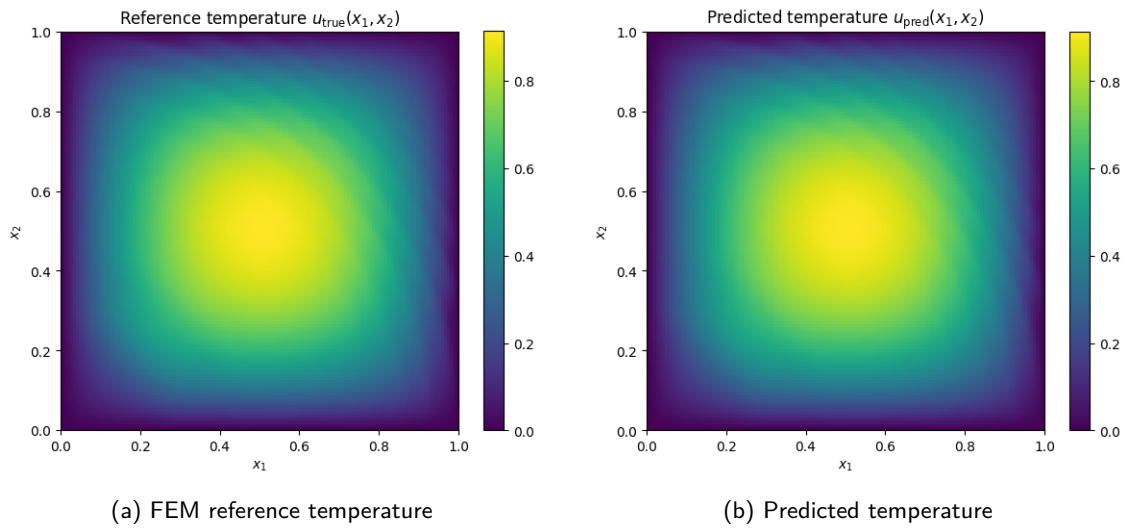


Figure 3.2: Comparison between the FEM reference temperature field and the FNO prediction for test sample 1 in Problem C.

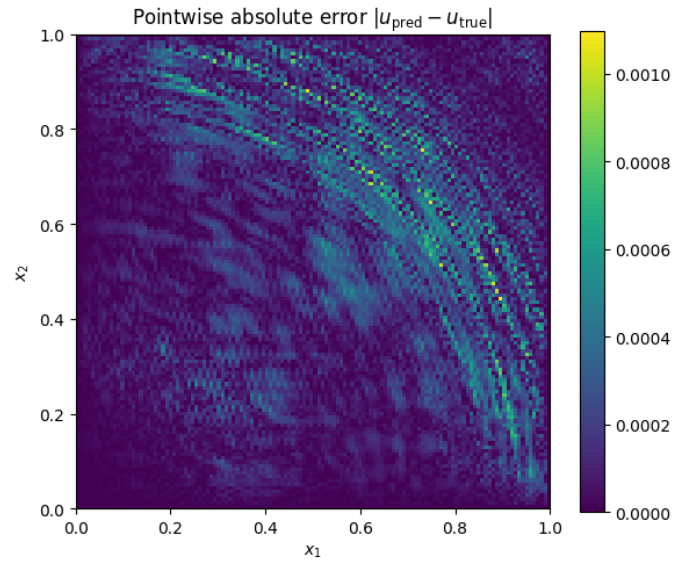


Figure 3.3: Pointwise absolute error  $|u_{\text{pred}} - u_{\text{FEM}}|$  for test sample 1 in Problem C.

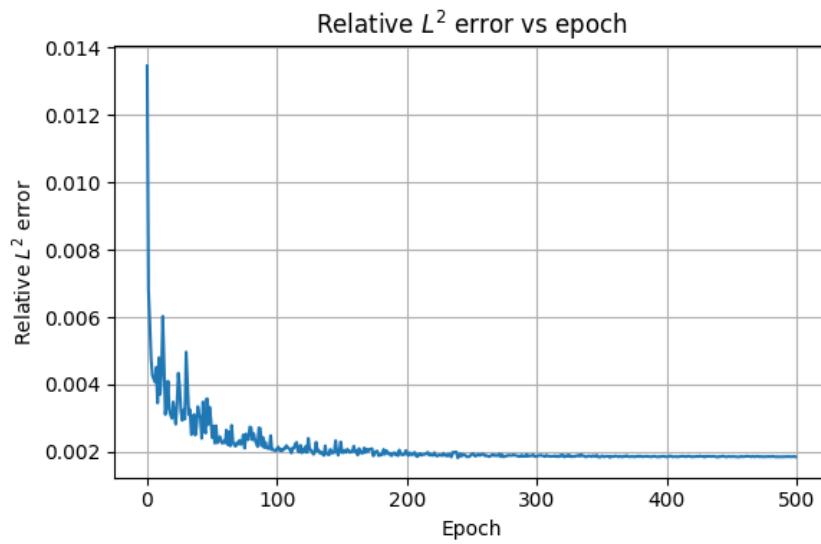


Figure 3.4: Mean relative  $L^2$  error over the 200 test samples during training for the final selected FNO model.

## Chapter 4

# Problem D

### 4.1 Problem Description

Problem D considers the prediction of traffic flow on the interval  $x \in (-1, 1)$  and  $t \in (0, 1)$ , where  $x$  denotes the spatial coordinate along the road segment and  $t$  denotes time. The unknown solution is the velocity field  $u(x, t)$  and the phenomena is governed by the viscous Burgers' equation

$$u_t + uu_x = \nu u_{xx}, \quad x \in (-1, 1), \quad t \in (0, 1), \quad (4.1)$$

where  $\nu = 0.1/\pi$  is the viscosity coefficient. The nonlinear term  $uu_x$  represents convection, while the term  $\nu u_{xx}$  represents diffusion.

Homogeneous Dirichlet boundary conditions are imposed at both ends of the spatial domain:

$$u(-1, t) = u(1, t) = 0, \quad t \in (0, 1). \quad (4.2)$$

The initial condition is given by

$$u(x, 0) = a(x), \quad (4.3)$$

where  $a(x)$  is the initial velocity profile. Therefore, for each initial condition  $a(x)$ , the solution of the Burgers' equation gives the corresponding full space-time velocity field  $u(x, t)$ .

The goal of Problem D is to learn the solution operator

$$G : a(x) \mapsto u(x, t).$$

This makes the problem an operator-learning task. The input is a one-dimensional initial field  $a(x)$ , while the output is a two-dimensional space-time field  $u(x, t)$ .

The dataset `ProblemD_dataset.h5` contains labelled, unlabelled, and test data. The labelled training data are `a_train_labeled` and `u_train_labeled`. The array `a_train_labeled` has shape  $(200, 256)$ , which means that there are 200 labelled initial conditions, each sampled at 256 spatial points. The corresponding solution array `u_train_labeled` has shape  $(200, 200, 256)$ , where the last two dimensions correspond to the 200 time steps and 256 spatial grid points.

The dataset also contains `a_train_unlabeled` with shape  $(1800, 256)$ . These data are additional initial conditions without the corresponding FDM solution fields. The test data are `a_test` and `u_test`, and are only used to evaluate the relative  $L^2$  error. The arrays `x_mesh` and `t_mesh` contain the spatial and temporal grid points.

## 4.2 Methodology

### 4.2.1 Method selection

Problem D is formulated as an operator-learning problem, where the model must learn the solution operator

$$G : a(x) \mapsto u(x, t),$$

which maps different initial velocity profiles to their corresponding space-time solutions. Therefore, conventional physics-informed neural networks, which normally approximate a single PDE solution, are not suitable. The relevant choices are deep neural operators such as DeepONet, Fourier Neural Operator, Physics-Informed DeepOnet and Physics-Informed Fourier Neural Operator [4, 2, 9].

The input initial condition  $a(x)$  and the output velocity field  $u(x, t)$  are both provided on regular grids. Therefore, FNO is chosen over DeepONet because it directly learns a full-field-to-full-field mapping through Fourier layers [2].

Physics-Informed FNO (PINO), is used instead of FNO because only 200 labelled pairs are available, while 1800 additional initial conditions are unlabelled. PINO uses the labelled samples through the data loss and the unlabelled samples through the Burgers' equation residual, allowing the complete training dataset to contribute [3, 9].

### 4.2.2 The loss function

Both supervised data and physics PDE loss contributes to the total training loss:

$$\mathcal{L} = w_{\text{data}}\mathcal{L}_{\text{data}} + w_{\text{PDE}}\mathcal{L}_{\text{PDE}}, \quad (4.4)$$

where  $w_{\text{data}} = 1.0$  and  $w_{\text{PDE}} = 0.1$ . The data loss is evaluated on the labelled data samples by comparing the predicted and reference velocity fields:

$$\mathcal{L}_{\text{data}} = \frac{1}{N_{\text{data}}} \sum |u_{\text{pred}} - u_{\text{true}}|^2. \quad (4.5)$$

For the unlabelled initial conditions data, no reference solution is available. Therefore, the predicted field is evaluated using the residual of the governing equation

$$r_{\theta} = \frac{\partial u_{\theta}}{\partial t} + u_{\theta} \frac{\partial u_{\theta}}{\partial x} - \nu \frac{\partial^2 u_{\theta}}{\partial x^2}, \quad (4.6)$$

and the PDE loss is defined as

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_r} \sum |r_{\theta}|^2. \quad (4.7)$$

The derivatives are approximated using central finite differences on the interior of the regular time-space grid. Each solution field contains  $200 \times 256$  grid points, and the PDE residual is evaluated at the  $198 \times 254$  interior points after excluding the first and last points in both directions.

The initial and boundary conditions are imposed using a hard constraints rather, resulting in no additional separate initial and boundary conditions loss terms and weights. The raw FNO output  $N_{\theta}(a, x, t)$  is transformed as

$$u_{\theta}(x, t) = a(x) + t(1 - x^2)N_{\theta}(a, x, t). \quad (4.8)$$

This ensures  $u_{\theta}(x, 0) = a(x)$  and, since the initial profiles satisfy the homogeneous boundary conditions,  $u_{\theta}(-1, t) = u_{\theta}(1, t) = 0$ .



### 4.2.3 The network structure

The PINO uses a two-dimensional Fourier Neural Operator acting on the space-time grid. For each sample, the network input has three channels,

$$[a(x), x, t],$$

where the initial condition  $a(x)$  is repeated along the temporal direction. The input therefore has shape  $200 \times 256 \times 3$ , while the network predicts the complete velocity field  $u(x, t)$  with shape  $200 \times 256 \times 1$ .

Firstly, the input is projected from three input channels to a hidden representation with width 32. The hidden features are then processed by four Fourier layers. Each layer consists of a spectral convolution, a pointwise linear transformation, and a GELU activation function. For the final model, 20 Fourier modes are retained in the temporal direction and 32 modes in the spatial direction. This choice was based on the additional hyperparameter trials presented in Appendix D. Lastly, two fully connected layers project the hidden features to the predicted velocity field. The remaining training hyperparameters, including the optimizer, learning-rate schedule, batch size, and number of epochs, are described in Section 4.3.1.

Table 4.1: Architecture of the PINO model for Problem D

| Layer              | Size / Channels           | Activation | Notes                            |
|--------------------|---------------------------|------------|----------------------------------|
| Input              | $200 \times 256 \times 3$ | –          | Channels: $a(x)$ , $x$ , and $t$ |
| Lifting layer      | $3 \rightarrow 32$        | –          | Pointwise linear projection      |
| Fourier layer 1    | $32 \rightarrow 32$       | GELU       | Spectral and pointwise paths     |
| Fourier layer 2    | $32 \rightarrow 32$       | GELU       | Spectral and pointwise paths     |
| Fourier layer 3    | $32 \rightarrow 32$       | GELU       | Spectral and pointwise paths     |
| Fourier layer 4    | $32 \rightarrow 32$       | GELU       | Spectral and pointwise paths     |
| Projection layer 1 | $32 \rightarrow 128$      | GELU       | Pointwise fully connected layer  |
| Output layer       | $128 \rightarrow 1$       | –          | Predicted velocity $u(x, t)$     |

### 4.2.4 The code link

[https://github.com/Manuel-Bryan-Arsenio/ED140023-Deep-Learning-for-PDEs-Project/blob/main/Problem\\_D\\_Results/Problem\\_D\\_Result\\_3\\_Final.ipynb](https://github.com/Manuel-Bryan-Arsenio/ED140023-Deep-Learning-for-PDEs-Project/blob/main/Problem_D_Results/Problem_D_Result_3_Final.ipynb)

## 4.3 Numerical Experiments

### 4.3.1 Experiment Setups

The final model was trained using 200 labelled and 1800 unlabelled samples. The labelled samples contain both the initial condition  $a(x)$  and their respective FDM solution  $u(x, t)$ , while the unlabelled samples contain only the initial condition. The 200 test samples were only used to compute the relative  $L^2$  error after each epoch. Each sample is defined on a  $200 \times 256$  time-space grid.

Before training, the initial-condition channel and velocity output were normalized using the mean and standard deviation of the training data. The coordinate channels  $x$  and  $t$  were not normalized. The model was trained using the Adam optimizer with an initial learning rate of  $10^{-3}$  and weight decay  $10^{-4}$ . A StepLR scheduler reduced the learning rate by a factor of 0.5 every 100 epochs. The labelled and unlabelled DataLoaders both used a batch size of 10,

so every training step combined one labelled batch for the data loss and one unlabelled batch for the PDE loss. The loss weights are chosen as

$$w_{\text{data}} = 1.0, \quad w_{\text{PDE}} = 0.5.$$

The final setup used 20 Fourier modes in the temporal  $t$  direction, 32 Fourier modes in the spatial  $x$  direction, and a hidden width of 32. This configuration was selected based on the additional trials presented in Appendix D.

Table 4.2: Numerical setup for training the PINO in PyTorch

| Parameter                   | Value                                      |
|-----------------------------|--------------------------------------------|
| Optimizer                   | Adam                                       |
| Initial learning rate       | $1 \times 10^{-3}$                         |
| Batch size                  | 10 labelled and 10 unlabelled samples      |
| Number of epochs            | 200                                        |
| Labelled training samples   | 200                                        |
| Unlabelled training samples | 1800                                       |
| Test samples                | 200                                        |
| Grid resolution             | $200 \times 256 = 51200$ points per sample |
| Fourier modes               | $m_t = 20, m_x = 32$                       |
| Hidden width                | 32                                         |
| Learning rate scheduler     | StepLR, step size 100, $\gamma = 0.5$      |
| Weight decay                | $1 \times 10^{-4}$                         |
| Hardware                    | NVIDIA RTX 4050 Laptop GPU (6GB)           |
| Precision                   | Float32                                    |
| Early stopping              | No                                         |
| Training time               | 6 h 59 min 39 s                            |

### 4.3.2 Numerical Results

The final PINO model was trained for 200 epochs on an NVIDIA RTX 4050 Laptop GPU. The total training time was 6 h 59 min 39 s. The final mean relative  $L^2$  error over the 200 unseen test samples was approximately

$$e_u \approx 1.56 \times 10^{-1}.$$

The test error decreased during training and nearly stabilizes on the final epochs. Compared with the additional trials in Appendix D, increasing the number of spatial Fourier modes improved the final accuracy.

The qualitative results for the first test sample are shown below. Figure 4.1 shows the unseen initial condition used as the input of the trained operator. The predicted velocity field in Figure 4.2 follows the FDM reference solution closely and captures the smoothing of the initial profile over time. As shown in Figure 4.3, the largest pointwise errors appear mainly during the early time steps and around regions with stronger spatial gradients. At later times, the error becomes smaller as the solution is smoothed by viscosity. While the pointwise absolute errors are relatively low, mostly in the order of  $10^{-2}$ , the higher mean relative  $L^2$  error over all test samples indicates that some unseen initial conditions are predicted less accurately than others.

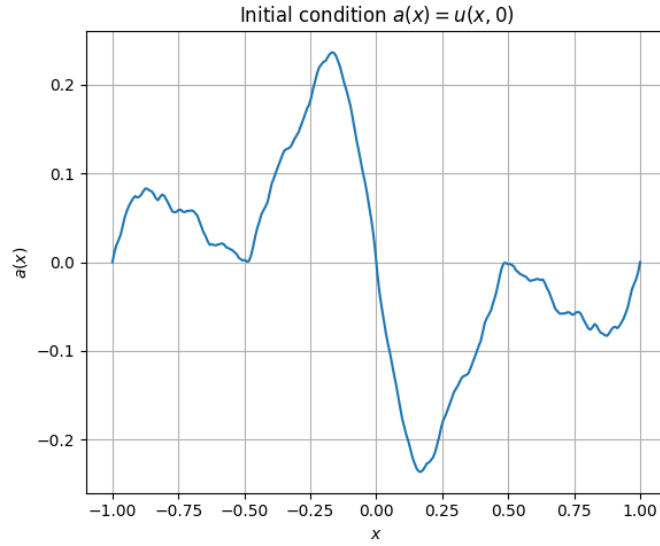
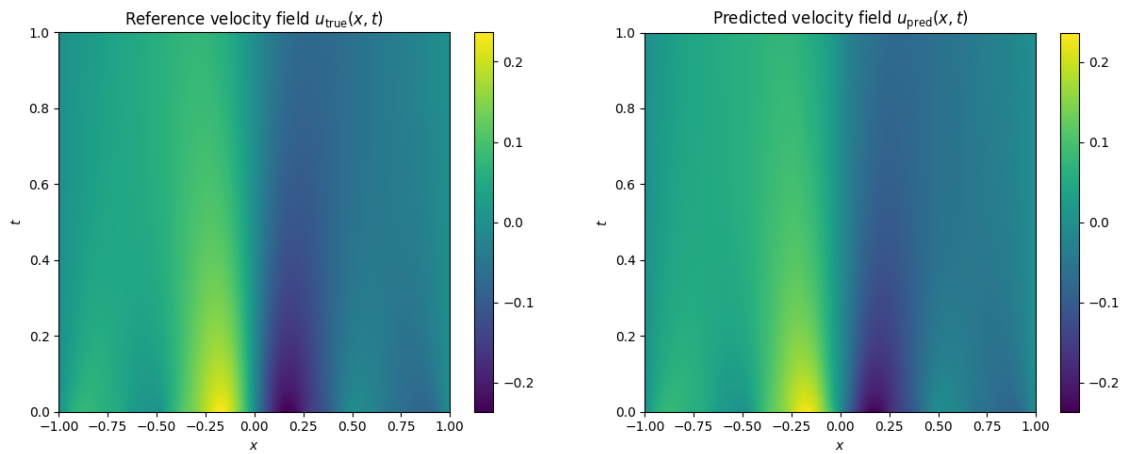


Figure 4.1: Initial condition  $a(x) = u(x, 0)$  for the first test sample in Problem D.



(a) FDM reference velocity field

(b) Predicted velocity field

Figure 4.2: Comparison between the FDM reference velocity field and the PINO prediction for the first test sample in Problem D.

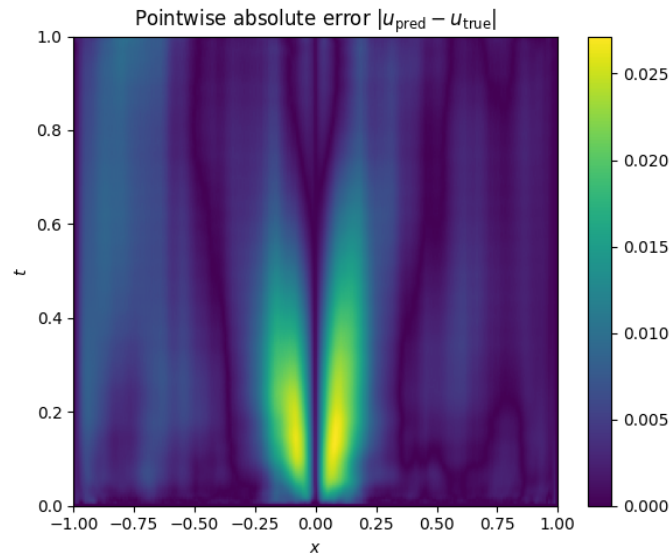


Figure 4.3: Pointwise absolute error  $|u_{\text{pred}} - u_{\text{true}}|$  for the first test sample in Problem D.

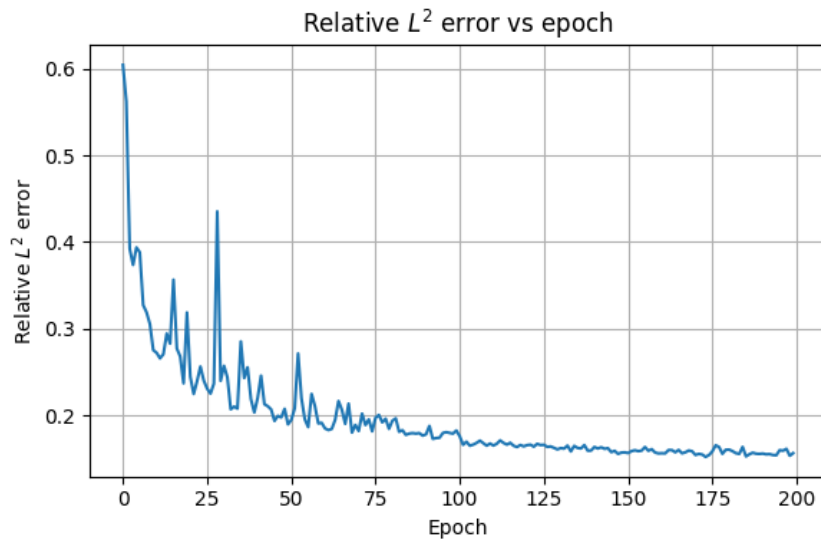


Figure 4.4: Mean relative  $L^2$  error over the 200 test samples during training for the final selected PINO model.

## Chapter 5

# Conclusions

This project showed that different PDE problems require different deep learning methods. For Problem A, a conventional inverse PINN was used to recover both the displacement field and the unknown Young's modulus. The displacement was recovered quite well, while the Young's modulus was more difficult because it had a more complex spatial variation. For Problem B, a VPINN was used because the weak formulation is more suitable for the discontinuous permeability field. However, the final result is still far from perfect, and especially the qualitative structure of the solution field was not captured very well. For Problem C, the FNO worked best because many labelled full-field samples were available on a regular grid. For Problem D, a PINO was used because only part of the training data was labelled, so the unlabelled initial conditions could still be used through the PDE residual.

One important thing learned from this project is that there is no single best method for every Problem. PINNs are flexible and useful for forward and inverse problems, but they can be difficult to train and are sensitive to the loss weights. Weak-form methods can reduce the derivative requirements, but the choice of test functions also becomes important. Neural operators are useful when many different PDE solutions need to be predicted, because after training they can give a full solution field with one forward pass. However, they still depend strongly on the training data and may not generalize well to inputs outside the training distribution.

The results also show that adding physics does not automatically make the prediction accurate. The physics loss can help when labelled data are limited, but it also makes the training more expensive. Even after almost 7 hours of training, the final relative  $L^2$  error for Problem D was still around  $1.56 \times 10^{-1}$ , so the result is still not close to perfect.

The main improvement would be to use a more strategic iteration plan for testing different hyperparameter settings. In this project, some runs took several hours, which limited how many combinations could be tested before the deadline. More powerful hardware would therefore also help significantly, especially for the operator-learning problems. For example, the final run for Problem D took 6 hours, 59 minutes, and 39 seconds on an NVIDIA RTX 4050 Laptop GPU. With faster hardware, more Fourier modes, larger widths, different loss weights, and longer training could be tested more systematically.

# References

- [1] E. Kharazmi, Z. Zhang, and G. E. Karniadakis. Variational physics-informed neural networks for solving partial differential equations. *arXiv preprint arXiv:1912.00873*, 2019.
- [2] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations*, 2021.
- [3] Z. Li, H. Zheng, N. Kovachki, D. Jin, H. Chen, B. Liu, K. Azizzadenesheli, and A. Anandkumar. Physics-informed neural operator for learning partial differential equations. *arXiv preprint arXiv:2111.03794*, 2021.
- [4] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis. Learning nonlinear operators via deeponet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3):218–229, 2021.
- [5] PyTorch Contributors. Softplus. <https://docs.pytorch.org/docs/stable/generated/torch.nn.Softplus.html>, 2026. Accessed 8 July 2026.
- [6] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational physics*, 378:686–707, 2019.
- [7] C. Rowan. On the failure of relu activation for physics-informed machine learning, 2025.
- [8] V. Scholz and Y. Zang. Fourier neural operator: Supervised learning for parametric pdes, part ii, 2026. Lecture slides, Deep Learning for PDEs in Engineering Physics, Technical University of Munich.
- [9] V. Scholz and Y. Zang. Physics-informed deep neural operators: Bridging data-driven operators and physics-informed learning, 2026. Lecture slides, Deep Learning for PDEs in Engineering Physics, Technical University of Munich.
- [10] M. Warzecha and Y. Zang. Deep neural operators for partial differential equations: Learning solution operators for parametric pdes, 2026. Lecture slides, Deep Learning for PDEs in Engineering Physics, Technical University of Munich.
- [11] Y. Zang and G. Bao. Particlelwnn: A novel neural networks framework for solving partial differential equations. *arXiv preprint arXiv:2305.12433*, 2023.
- [12] Y. Zang, G. Bao, X. Ye, and H. Zhou. Weak adversarial networks for high-dimensional partial differential equations. *Journal of Computational Physics*, 411:109409, 2020.

## Appendix A

# Additional Hyperparameter Trials for Problem A

This appendix summarizes selected preliminary experiments performed for Problem A. These trials were used to choose a stable final configuration for the inverse PINN.

Table A.1: Summary of selected hyperparameter trials for Problem A. The final reported model is shown in the last row.

| Run      | $k_\phi$ architecture  | Epochs | Loss weights | $e_u$                 | $e_k$                 |
|----------|------------------------|--------|--------------|-----------------------|-----------------------|
| Result 1 | [1, 40, 40, 40, 40, 1] | 10000  | (2,200)      | $5.68 \times 10^{-3}$ | $1.47 \times 10^{-1}$ |
| Result 2 | [1, 40, 40, 40, 40, 1] | 20000  | (20,200)     | $7.07 \times 10^{-3}$ | $1.67 \times 10^{-1}$ |
| Result 3 | [1, 64, 64, 64, 64, 1] | 5000   | (2,200)      | $6.56 \times 10^{-3}$ | $1.48 \times 10^{-1}$ |
| Result 4 | [1, 64, 64, 64, 64, 1] | 20000  | (2,200)      | $4.15 \times 10^{-3}$ | $8.67 \times 10^{-2}$ |
| Final    | [1, 64, 64, 64, 64, 1] | 25000  | (2,200)      | $3.93 \times 10^{-3}$ | $8.11 \times 10^{-2}$ |

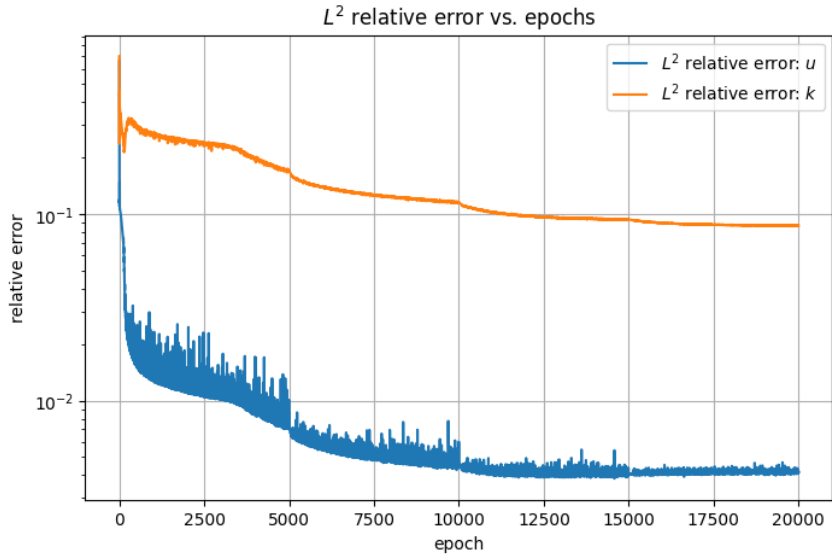
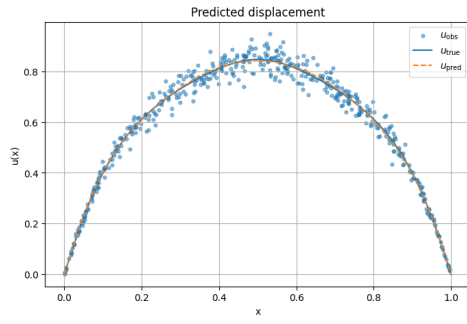
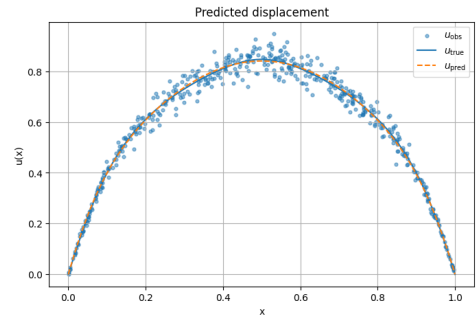


Figure A.1: Relative  $L^2$  error history for a preliminary 20000-epoch run. The coefficient error  $e_k$  was still decreasing slowly near the end of training, motivating the longer 25000-epoch final run.

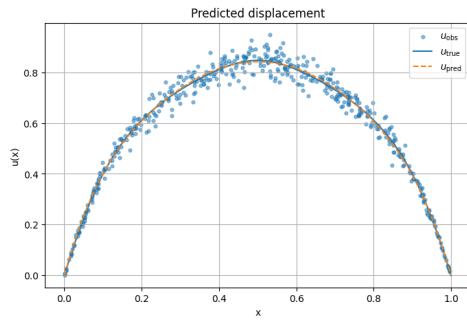
The final configuration was selected based on the accuracy of the recovered field.



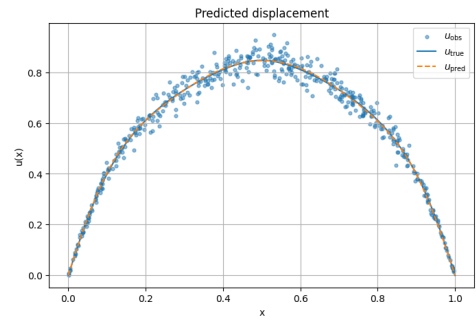
(a) Run 1



(b) Run 2

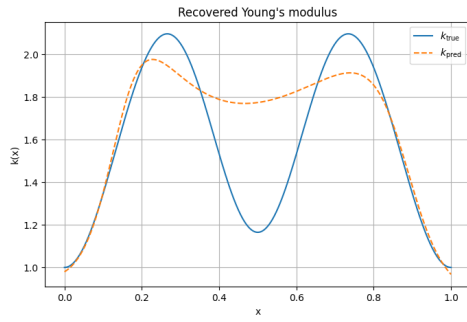


(c) Run 3

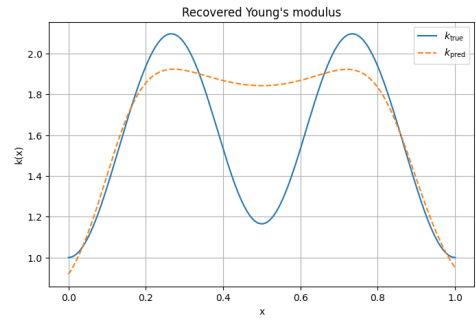


(d) Run 4

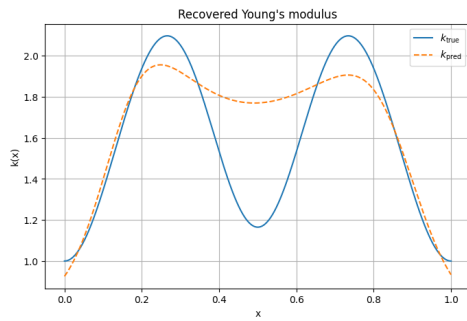
Figure A.2: Predicted displacement fields from the other inverse PINN runs for Problem A.



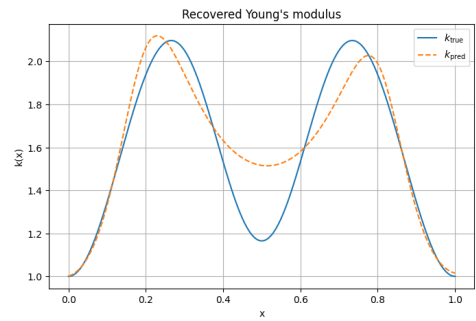
(a) Run 1



(b) Run 2



(c) Run 3



(d) Run 4

Figure A.3: Recovered Young's modulus from the other inverse PINN runs for Problem A.



## Appendix B

# Additional Hyperparameter Trials for Problem B

Several preliminary VPINN runs were tested before selecting the final setup for Problem B. The main parameters varied were the network size, the number of Legendre test functions, the loss weights, the batch size, and the boundary-condition treatment. The final relative  $L^2$  errors are summarized in Table B.1.

Table B.1: Summary of additional VPINN runs for Problem B

| Run | Architecture               | $N_{\text{test}}$ | Batch | BC   | $w_{\text{PDE}}$ | $w_{\text{BD}}$ | $e_u$                 |
|-----|----------------------------|-------------------|-------|------|------------------|-----------------|-----------------------|
| 1   | [2, 40, 40, 40, 40, 1]     | 5                 | 1000  | Soft | 2                | 10              | $1.55 \times 10^{-1}$ |
| 2   | [2, 64, 64, 64, 64, 1]     | 5                 | 1000  | Soft | 2                | 10              | $1.71 \times 10^{-1}$ |
| 3   | [2, 64, 64, 64, 64, 64, 1] | 7                 | 1000  | Soft | 2                | 10              | $1.46 \times 10^{-1}$ |
| 4   | [2, 64, 64, 64, 64, 1]     | 5                 | 1000  | Soft | 1                | 50              | $9.16 \times 10^{-2}$ |
| 5   | [2, 64, 64, 64, 64, 1]     | 6                 | 1000  | Soft | 1                | 50              | $8.61 \times 10^{-2}$ |
| 6   | [2, 64, 64, 64, 64, 1]     | 7                 | 1000  | Soft | 1                | 50              | $9.37 \times 10^{-2}$ |
| 7   | [2, 64, 64, 64, 64, 1]     | 6                 | 4096  | Soft | 1                | 50              | $1.00 \times 10^{-1}$ |
| 8   | [2, 64, 64, 64, 64, 1]     | 6                 | 1000  | Soft | 2                | 50              | $9.56 \times 10^{-2}$ |
| 9   | [2, 64, 64, 64, 64, 1]     | 6                 | 1000  | Hard | 1                | —               | $1.02 \times 10^{-1}$ |
| 10  | [2, 64, 64, 64, 64, 1]     | 6                 | 1000  | Hard | 1                | —               | $1.03 \times 10^{-1}$ |

The predicted pressure fields from the additional runs are shown in Figure B.1. Run 5 was selected as the final setup because it achieved the lowest final relative  $L^2$  error among the tested configurations, while still keeping the computational cost reasonable. Compared with the other tested settings, the  $N_{\text{test}} = 6$  setup provided a good balance between solution accuracy, training stability, and the number of weak test functions. Therefore, Run 5 was retained as the final VPINN configuration.

Although the runs imposed with hard boundary constraint (i.e., run 9 and run 10), show visible local structures in the predicted pressure field, they do not give the lowest relative  $L^2$  error. This indicates that the additional local variation is not necessarily closer to the reference solution over the full domain. Therefore, the final setup was selected based on both the quantitative error in Table B.1 and the qualitative pressure-field comparison in Figure B.1.

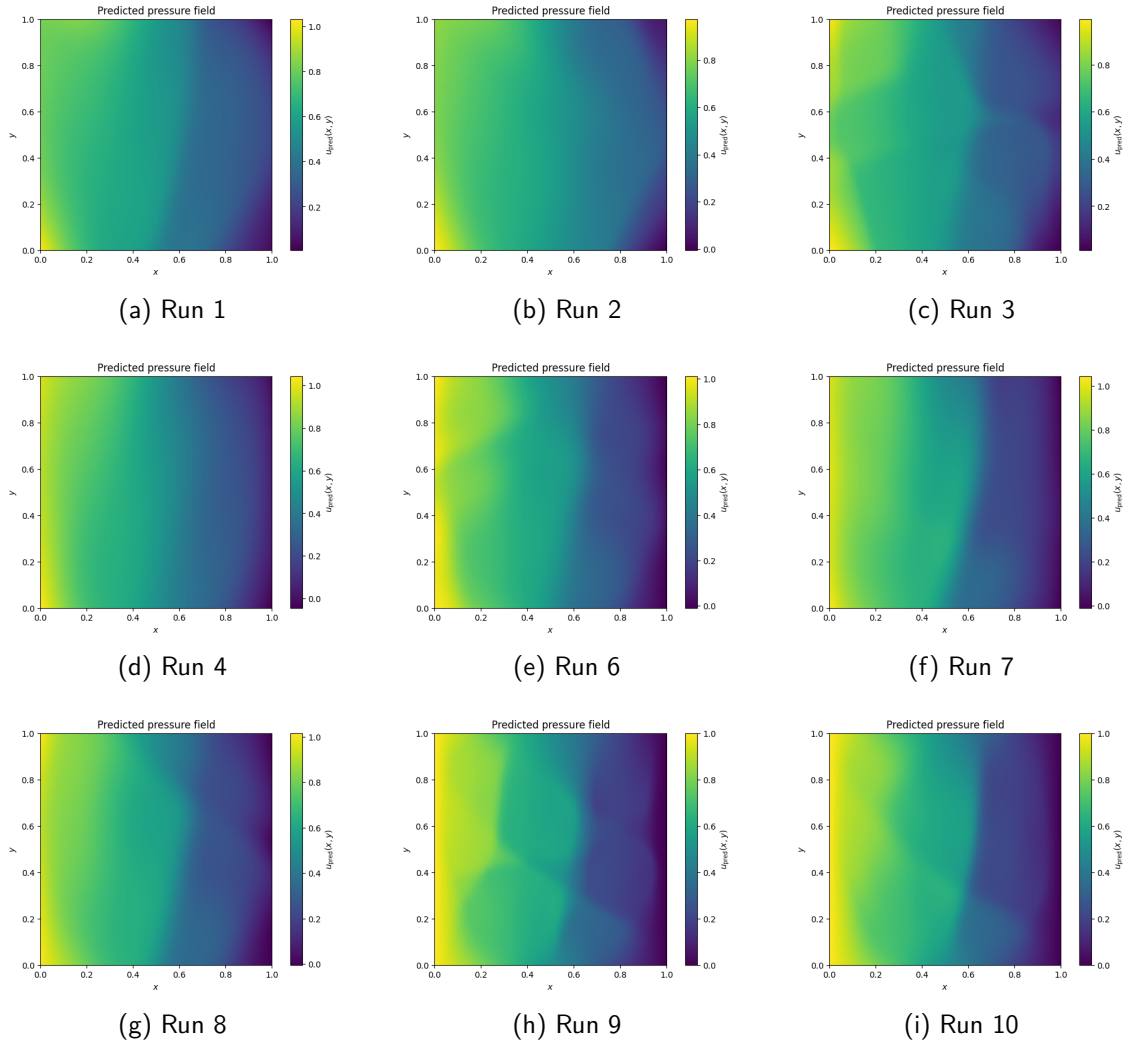


Figure B.1: Predicted pressure fields from other VPINN runs for Problem B.

## Appendix C

# Additional Hyperparameter Trials for Problem C

Several additional FNO runs were performed for Problem C to study the effect of the number of Fourier modes and the hidden width. All runs used the same supervised FNO setup, with 1000 training samples, 200 test samples, batch size 10, learning rate  $10^{-3}$ , Adam optimizer, StepLR scheduler, and 500 training epochs.

Table C.1: Summary of additional FNO runs for Problem C

| Run | $m_1$ | $m_2$ | Width | Epochs | Runtime         | Final loss                | $e_u$                     |
|-----|-------|-------|-------|--------|-----------------|---------------------------|---------------------------|
| 1   | 12    | 12    | 32    | 500    | 50 min 59 s     | $1.286282 \times 10^0$    | $3.649275 \times 10^{-3}$ |
| 2   | 16    | 16    | 32    | 500    | 55 min 50 s     | $9.270835 \times 10^{-1}$ | $2.796658 \times 10^{-3}$ |
| 3   | 12    | 12    | 48    | 500    | 3 h 8 min 52 s  | $9.840469 \times 10^{-1}$ | $3.519219 \times 10^{-3}$ |
| 4   | 20    | 20    | 32    | 500    | 1 h 37 min 38 s | $6.944903 \times 10^{-1}$ | $1.855718 \times 10^{-3}$ |

The results show that increasing the number of Fourier modes was more effective than increasing the hidden width. Increasing the modes from 12 to 16, and then to 20, reduced the test error clearly. On the other hand, increasing the width from 32 to 48 caused a much larger runtime increase, while the error reduction is smaller compared to increasing the Fourier modes. Therefore, the final reported model uses 20 Fourier modes in both spatial directions and a hidden width of 32.

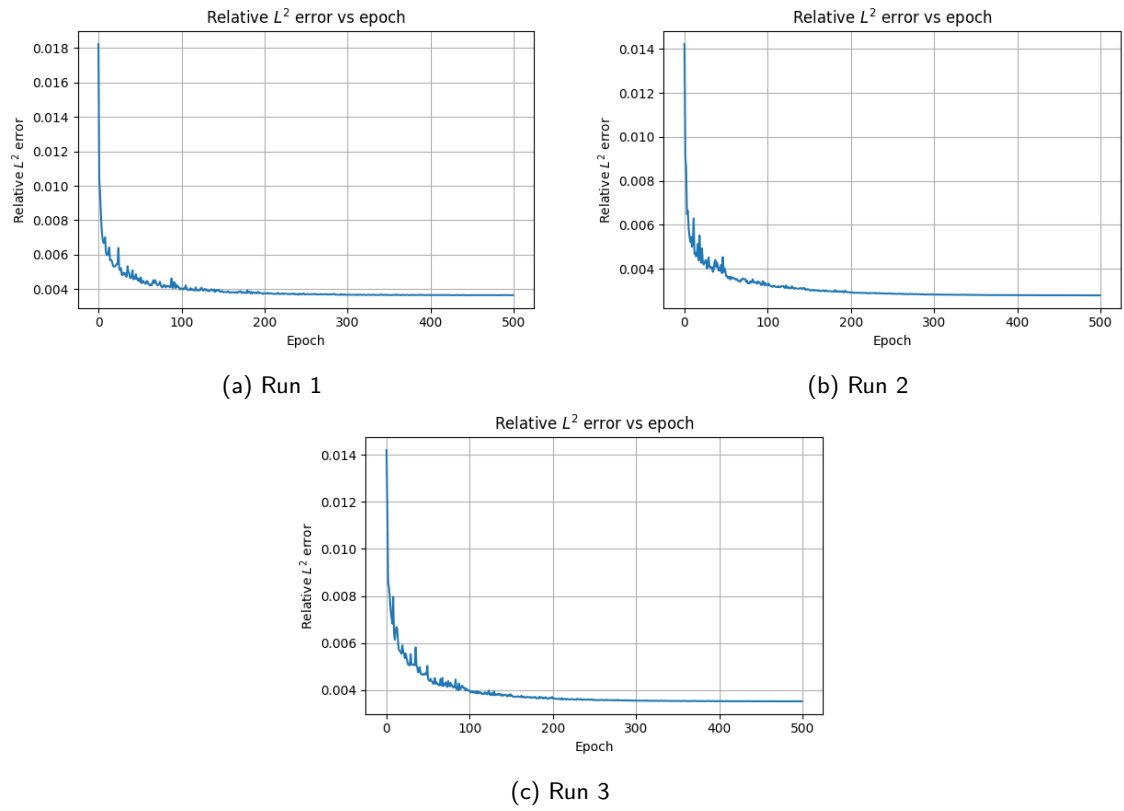


Figure C.1: Relative  $L^2$  error over training epochs from other FNO runs for Problem C.

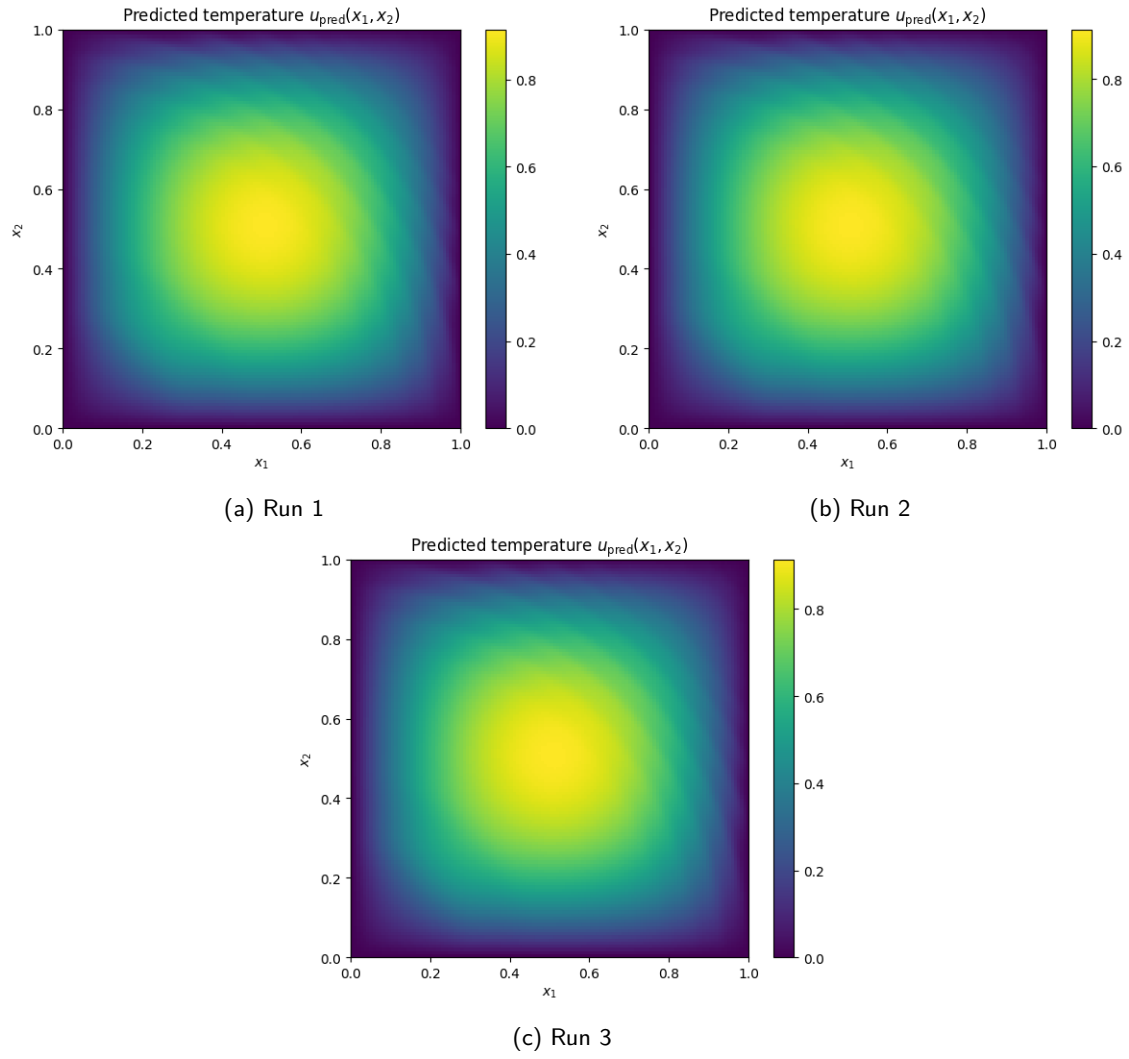


Figure C.2: Predicted temperature fields for the first test sample from other FNO runs for Problem C.

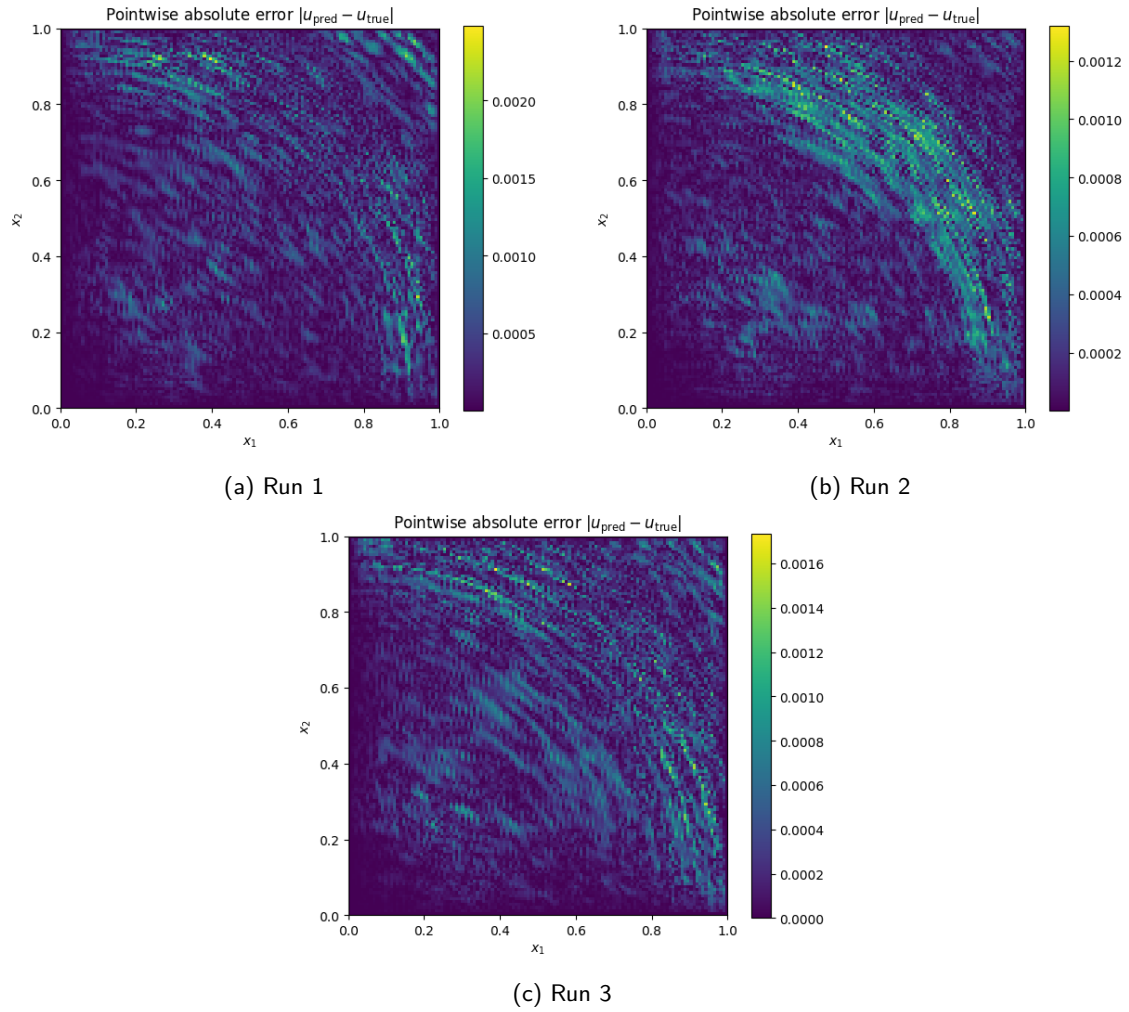


Figure C.3: Pointwise absolute error  $|u_{\text{pred}} - u_{\text{FEM}}|$  for the first test sample from other FNO runs for Problem C.

## Appendix D

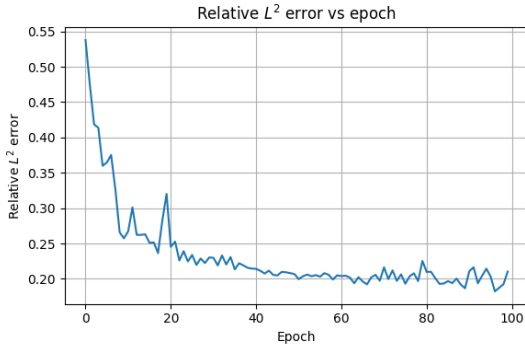
# Additional Hyperparameter trials for Problem D

Three PINO runs were performed for Problem D to study the influence of the number of Fourier modes, the PDE loss weight, and the number of training epochs. All runs used a hidden width of 32, batch size 10, learning rate  $10^{-3}$ , Adam optimizer, and the same hard enforcement of the initial and boundary conditions.

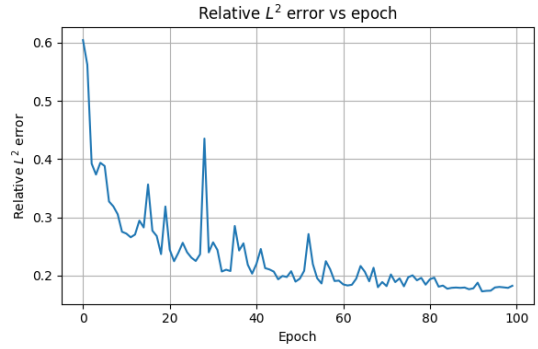
Table D.1: Summary of PINO runs for Problem D

| Run | $m_t$ | $m_x$ | Width | Epochs | Step size | $w_{\text{data}}$ | $w_{\text{PDE}}$ | $e_u$                         |
|-----|-------|-------|-------|--------|-----------|-------------------|------------------|-------------------------------|
| 1   | 20    | 20    | 32    | 100    | 50        | 1.0               | 0.5              | $\approx 2.10 \times 10^{-1}$ |
| 2   | 20    | 32    | 32    | 100    | 100       | 1.0               | 0.5              | $\approx 1.82 \times 10^{-1}$ |
| 3   | 20    | 32    | 32    | 200    | 100       | 1.0               | 0.5              | $\approx 1.56 \times 10^{-1}$ |

Increasing the number of spatial Fourier modes from 20 to 32 reduced the test error, indicating that the spatial representation was initially limiting the model accuracy.



(a) Run 1:  $m_t = 20$ ,  $m_x = 20$ ,  $w_{\text{PDE}} = 0.5$



(b) Run 2:  $m_t = 20$ ,  $m_x = 32$ ,  $w_{\text{PDE}} = 0.5$

Figure D.1: Relative  $L^2$  error vs epochs from other PINO runs for Problem D.

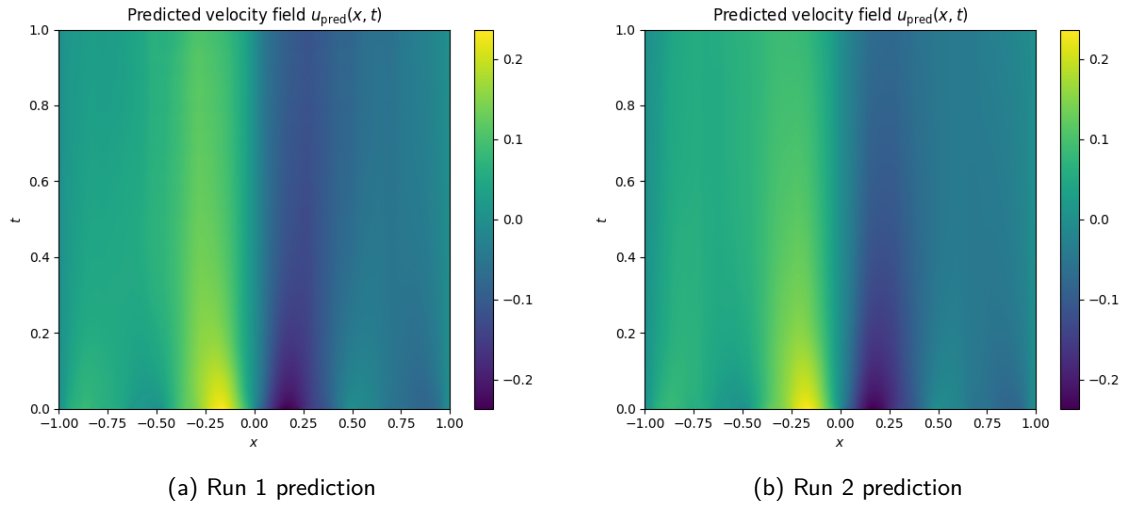


Figure D.2: Predicted velocity fields for the first test sample from other PINO runs for Problem D.

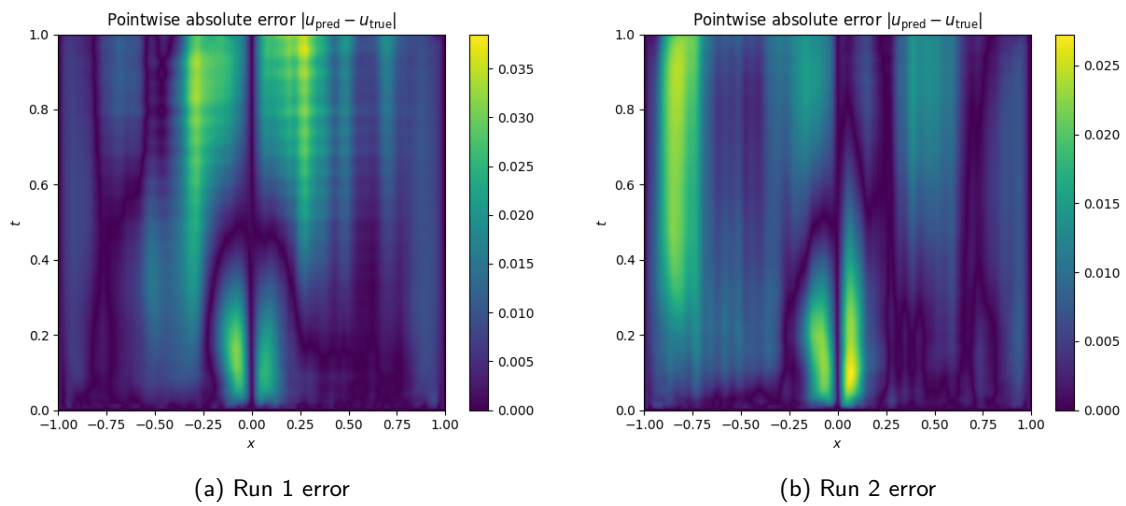


Figure D.3: Pointwise absolute error  $|u_{\text{pred}} - u_{\text{true}}|$  for the first test sample from other PINO runs for Problem D.